



Outage Detection in Web Traffic Using Statistical Models and Neural Networks

Bachelor's Thesis

Student: Antonio Cabrera Landín

Supervisor: Stanislav Vakaruk

Degree: Software Engineering

Note on this translation

This document is an English translation of my bachelor's thesis in Software Engineering, *Detección de caídas en el tráfico web mediante modelos estadísticos y redes neuronales*, written in Spanish and submitted at U-tad (Madrid) in 2026. It was translated from the Spanish original with the help of an AI model. The abstract is the English version I wrote for the original thesis, and the figures keep their original Spanish labels. In case of any discrepancy, the Spanish original is the authoritative version.

Antonio Cabrera Landín

Acknowledgments

I would like to thank my family for their support throughout these five years of my degree, and all the professors who accompanied me, for everything they have taught me.

Contents

List of Figures	3
List of Tables	5
Abstract	6
Glossary	7
Notation	9
1 Introduction	10
1.1 Justification and context	10
1.2 Problem statement	10
1.3 Objectives of the thesis	11
2 State of the Art	12
2.1 Historical evolution	12
2.2 Theoretical framework of the thesis	13
2.3 Related work	37
3 Methodology	39
3.1 Methodology	39
3.2 Technologies used	42
4 Development	45
4.1 Data preparation and preprocessing	45
4.2 Exploratory data analysis	52
4.3 Common evaluation framework	54
4.4 Models	56
4.5 Hyperparameter tuning	67
4.6 Model evaluation	68
4.7 Real-time detection application	71

5	Conclusions	73
5.1	Conclusions	73
5.2	Future work	73
	References	75

List of Figures

2.1	Classical decomposition of a time series.	16
2.2	Example of a stationary and a non-stationary time series.	17
2.3	ACF and PACF plots of a time series.	18
2.4	Additive outlier.	22
2.5	Innovational outlier.	23
2.6	Level-shift outlier.	23
2.7	Temporary-change outlier.	24
2.8	Example of a neural network architecture.	26
2.9	ReLU function.	27
2.10	Sigmoid function.	28
2.11	Hyperbolic tangent function.	29
2.12	Diagram of an LSTM cell. Source: Zhang et al., 2023.	30
2.13	Architecture of a CNN.	34
2.14	Architecture of VGG-16. Source: github.com/spmallick/learnopencv	35
2.15	Architecture of DeepAnT. Source: Munir et al., 2019.	36
4.1	Time series of the data.	46
4.2	Examples of total and partial outages.	48
4.3	Incident viewer.	49
4.4	Distribution of outage types.	49
4.5	Comparison of query request volume between a working Wednesday and a holiday Wednesday.	50
4.6	Chronological split of the dataset into training, validation, and test sets. The vertical red bands indicate the annotated incidents.	52
4.7	Typical daily pattern of query traffic.	52
4.8	Scheduled maintenance from Friday to Monday.	53
4.9	Traffic for a representative day after the structural change (with no recorded incidents).	54
4.10	Additive seasonal decomposition of the series with period $m = 48$	58
4.11	ACF and PACF correlograms of the series up to 72 lags.	58
4.12	Actual values and SARIMAX predictions.	60

4.13	Architecture of the implemented DeepAnT model.	61
4.14	Evolution of the training and validation errors during DeepAnT training.	62
4.15	Actual values and DeepAnT predictions.	63
4.16	Architecture of the implemented LSTM model.	64
4.17	Training curves of a preliminary LSTM run with 200 fixed epochs.	65
4.18	Training curves of the LSTM with early stopping enabled.	66
4.19	Actual values and LSTM predictions over a week of the test set.	67
4.20	Optuna chart showing the importance of each hyperparameter.	67
4.21	Detections of the four models over a day with multiple incidents from the test set. .	69
4.22	Dashboard of the real-time detection application.	72

List of Tables

4.1	Dataset structure.	45
4.2	Structure of the processed incidents.	47
4.3	Dataset structure after preprocessing.	51
4.4	SARIMAX prediction metrics on the test set.	59
4.5	DeepAnT prediction metrics on the test set.	63
4.6	LSTM prediction metrics on the test set.	66
4.7	Detection metrics of the models on the test set.	68
4.8	Median detection latency and total false-alarm time of the models on the test set. . .	69
4.9	NAB score of various reference detectors on Numenta’s official benchmark (Ahmad et al., 2019).	70

Abstract

Development of an application-outage alerting system based on anomaly detection over traffic time series. Real production data from 2023 to 2025 are used, belonging to a nationwide application whose existing rule-based alarm system generates an excessive number of false positives. After a manual selection, 355 incidents covering total and partial outages were labeled and used as the ground truth on which the models are evaluated.

Four detectors are implemented under identical conditions: a heuristic baseline, a SARIMAX model, the *DeepAnT* convolutional network and an LSTM recurrent network. The comparison over the test set places the LSTM as the most accurate detector ($F_1 = 0.675$ and NAB score of 65.53), closely followed by DeepAnT. The final deliverable integrates the three best-performing models (baseline, DeepAnT and LSTM) into a real-time monitoring system with a dashboard interface, applying a consensus criterion among detectors that reduces individual false positives.

Keywords: anomaly detection, time series, SARIMAX, CNN, LSTM.

Glossary

ACF	Autocorrelation Function
PACF	Partial Autocorrelation Function
WN	White Noise
SR	Spectral Residual
AR	Autoregressive
MA	Moving Average
ARMA	Autoregressive Moving Average
ARIMA	Autoregressive Integrated Moving Average
SARIMA	Seasonal Autoregressive Integrated Moving Average
SARIMAX	Seasonal Autoregressive Integrated Moving Average with eXogenous factors
AO	Additive Outlier
IO	Innovational Outlier
LS	Level Shift
TC	Temporary Change
KNN	k-Nearest Neighbors
LOF	Local Outlier Factor
DNN	Deep Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
CNN	Convolutional Neural Network
SGD	Stochastic Gradient Descent
AIC	Akaike Information Criterion
ReLU	Rectified Linear Unit
ADF	Augmented Dickey-Fuller
KPSS	Kwiatkowski-Phillips-Schmidt-Shin

TP	True Positives
FN	False Negatives
FP	False Positives
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
MAPE	Mean Absolute Percentage Error
sMAPE	Symmetric Mean Absolute Percentage Error
NAB	Numenta Anomaly Benchmark
EDA	Exploratory Data Analysis
API	Application Programming Interface

Notation

$E(X)$	Expected value of a random variable X
$\text{Var}(X)$	Variance of a random variable X
$X \sim \text{WN}(\mu, \sigma^2)$	Random variable X following a white noise distribution with mean μ and variance σ^2
X_t	Time series indexed by time t
x_t	Observation of the series at time t
$\hat{x}_t$	Forecast of the series at time t
a_t	White noise term at time t
γ_k	Autocovariance of the series at lag k
ρ_k	Autocorrelation of the series at lag k
(p, d, q)	Autoregressive, integration, and moving average orders of the ARIMA model
$(P, D, Q)_m$	Seasonal orders of the SARIMA model with period m
ϕ_i	Autoregressive coefficients of the ARIMA model
θ_j	Moving average coefficients of the ARIMA model
ω_k	Coefficients associated with the exogenous variables
W	Size of the historical input window of the neural networks
L	Multi-step forecast horizon of the LSTM network
F_1	F_1 score
R^2	Coefficient of determination

Chapter 1

Introduction

This chapter presents the general context of the thesis. First, the relevance of the outage detection problem in real applications is justified and placed within the current context of the software industry. Next, the specific problem that motivates this thesis is explained, associated with a real case of a company with a rudimentary alarm system. Finally, the specific objectives proposed to address this problem are listed.

1.1 Justification and context

One of the longstanding priorities of the software industry has been maintaining the availability and performance of its services. A system outage can cause critical losses, not only for the service provider but for all its customers. A recent example is the CrowdStrike incident. According to the impact analysis by the firm Parametrix (Parametrix, 2024), the event is estimated to have caused losses of 5,400 million dollars for Fortune 500 companies.

As a result, engineers have implemented various techniques to detect, investigate the cause of, and even prevent this type of outage. Historically, static statistical parameters have been used as a baseline; however, the exponential growth in data volumes and the adoption of Big Data in the sector have created a need for more sophisticated methods to detect these outliers.

1.2 Problem statement

The problem is based on a real case involving a company whose application provides service at a national level. This company wants to know whether its servers are being attacked or are out of service, and to that end it developed an alarm system based on the number of requests in the traffic. The problem stems from the number of false positives produced by the monitoring system: the current rule for deciding whether traffic is anomalous is very rudimentary and therefore unreliable.

This thesis sets out to solve this problem using different extreme value detection models, ranging from classical statistical models to more modern techniques based on Machine Learning.

The model will need to account for the inherent complexity of the traffic data. First, it will need to consider annual and weekly seasonality, as well as reduced traffic on public holidays. In addition, it will have to capture the daily activity pattern, marked by reduced traffic at night and pronounced peaks during working hours.

1.3 Objectives of the thesis

The final goal of the thesis is to develop a real-time anomaly detection and visualization system for the application's request traffic. This system will need to detect service outages using one or more outlier detection models trained on the request time series. These models will report extreme values taking into account the time of day and year, as well as whether each day is a working day or a public holiday. The final application will include a visual interface that enables monitoring of server status. To achieve this final goal, the following objectives are proposed:

- 1) Clean and sanitize the dataset, addressing duplicate values and possible null values, and standardizing the data type of each column.
- 2) Perform an exploratory analysis of the data. This involves studying the health and quality of the data, as well as visualizing various charts to understand the nature of the time series.
- 3) Manually label the incidents present in the dataset, in order to have a ground truth against which to evaluate the models.
- 4) Evaluate different techniques for detecting outliers in time series and draw conclusions about their advantages and disadvantages for this particular case.
- 5) Implement different models for detecting outliers, both statistical and based on Machine Learning.
- 6) Tune the hyperparameters of the implemented models using an automated search procedure.
- 7) Study the performance of each model using anomaly detection metrics such as the F_1 score and the NAB score.
- 8) Integrate the models into a real-time monitoring application that displays the data stream and alerts on detected anomalies through a dashboard.
- 9) Validate and verify the final solution, ensuring that the program meets the above requirements.

Chapter 2

State of the Art

This chapter situates the thesis within the scientific literature on outlier detection in time series. First, a historical overview is given of the evolution of detection techniques, from the classical criteria of the 19th century to modern deep learning architectures. Next, the theoretical framework introduces the mathematical concepts and algorithms underlying the development of the thesis. Finally, recent publications addressing similar problems in the field of network traffic anomaly detection are examined.

2.1 Historical evolution

The problem of outliers goes back a long way in the history of statistics. Surveys conducted in the 18th and 19th centuries already showed that the problem was known to mathematicians (Hawkins, 1980, p. 9).

In the 18th century, researchers' most common solution was to eliminate observations that seemed out of place. This rudimentary method was already harshly criticized by the mathematician Bernoulli (Hawkins, 1980, p. 10).

In the 19th century, more elaborate techniques for detecting outliers emerged from astronomy. In 1852, Benjamin Peirce proposed the first objective criterion for rejecting outliers, which screened observations based on whether they exceeded a certain probabilistic threshold (Hawkins, 1980, p. 10). Later, in 1866, the mathematician Chauvenet formulated a simpler criterion based on the normal distribution (Hawkins, 1980, p. 10).

The 20th century saw the birth of outlier detection methods for temporal data. In 1970, the Box-Jenkins methodology was put forward (Box et al., 2008), revolutionizing time series research. Using the Box-Jenkins models, in 1972, A. J. Fox presented two models for detecting outliers in time series (Fox, 1972). Five years later (1977), John W. Tukey published his book *Exploratory Data Analysis* (Tukey, 1977), in which he popularized the box plot and the interquartile range rule for detecting outliers. Two decades later, automated algorithms were developed to detect and minimize the effect

of outliers (Chang et al., 1988; Chen & Liu, 1993).

At the end of the century (1998), researchers Edwin Knorr and Raymond Ng presented their distance-based detection approach. Rather than relying on a probability distribution, they evaluated the distances between each observation's neighbors (Knorr & Ng, 1998, p. 393). The advantage of this approach is that the proposed algorithms were much more computationally efficient compared to previous methods (Knorr & Ng, 1998, p. 392). In the following century (2000), this approach was refined with the publication of the LOF algorithm (*Local Outlier Factor*) (Breunig et al., 2000). This algorithm solves the problem of data clusters with different densities (Aggarwal, 2017, p. 131).

In 2008, an important paradigm shift took place. Instead of modeling what an anomalous data point is, it was proposed to isolate outliers using unsupervised learning (Liu et al., 2008, abstract). Specifically, the *Isolation Forest* algorithm was presented, which reduces computation to linear complexity and also makes it possible to work with high-dimensional data (Liu et al., 2008, p. 414).

In 2015, several companies released the source code of their detection systems. First, Twitter revealed the mathematical technique behind its R package (*AnomalyDetection*) (Kejariwal et al., 2015). Similarly, Yahoo described how EGADS works, an automated large-scale outlier detection system for time series (Laptev et al., 2015). Likewise, Netflix opened the source code of its outlier detection function for Big Data (Wong et al., 2015).

That same year, researchers began using recurrent neural networks (RNNs), specifically the LSTM architecture, to detect anomalies in temporal data (Malhotra et al., 2015). To do this, they trained the model on data free of outliers, with the aim of using prediction errors to model the occurrence of outliers.

Finally, in 2018, DeepAnT was presented. This methodology uses deep convolutional neural networks to find anomalies (Munir et al., 2019). Unlike LSTM-based models, DeepAnT uses unsupervised learning, allowing the use of raw, unlabeled data. In addition, it requires less data than its predecessors, since only 40% of the sample was used to train it (Munir et al., 2019, p. 1992).

2.2 Theoretical framework of the thesis

This section lays out the theoretical concepts necessary for the subsequent development of this thesis. In addition, the most relevant outlier detection algorithms will be explained.

2.2.1 Extreme value (Outlier)

An outlier (also known as an atypical value, extreme value, or anomaly) is an observation that differs significantly from the data. In the literature, it is common to use Hawkins's definition: "An

observation which deviates so much from other observations as to arouse suspicions that it was generated by a different mechanism” (Hawkins, 1980, p. 1).

Some of the most common causes of these anomalous data points are: measurement and recording errors, and unusual events such as strikes, natural disasters, or sudden changes in laws and policies.

Although they have historically been perceived as erroneous data that degraded prediction models, they often contain valuable information about the behavior of the system. For example, the study of outliers helps detect diseases, detect intrusions in computer systems, or detect criminal activity (Aggarwal, 2017, pp. 1–2).

2.2.2 Stochastic process

A stochastic process is a statistical phenomenon that evolves over time according to probabilistic laws (Box et al., 2008, p. 22). Mathematically, it is defined as a sequence of random variables, chronologically ordered and equally spaced (Brockwell & Davis, 2016, p. 6).

$$X_0, X_1, X_2, \dots \text{ where } X_t \text{ is a random variable.}$$

These stochastic processes are used to model the behavior of real observations and to make predictions. In addition, they make it possible to detect anomalous values.

Some examples of these processes are Markov chains, which can be used to suggest text on a virtual keyboard, or the random walk, which is used to model the stock market in a rudimentary way.

2.2.3 Time series

A time series is a set of observations x_t , each recorded at a specific time t (Brockwell & Davis, 2016, p. 1). Examples of time series include a country’s macroeconomic indicators or the planet’s average annual temperatures.

It can also be defined as a sample period of a stochastic process (Box et al., 2008, p. 23).

Typically, the observed data are not independent of each other. Analyzing this dependence allows us to understand the behavior of the data or even make predictions. (Box et al., 2008, p. 1)

Types of time series

There are different types of time series:

A time series can be continuous or discrete: discrete series collect their data at specific points in time, while continuous series record observations continuously over a time interval (Brockwell & Davis,

2016, p. 1). This thesis will study discrete time series whose data have been collected at fixed time intervals.

A time series can be univariate or multivariate: when the observations correspond to a single variable, we say the series is univariate. When they correspond to more than one, we say it is multivariate.

2.2.4 Classical decomposition model

In order to analyze time series, the classical approach (Brockwell & Davis, 2016, sec. 1.5) consists of decomposing the series into different components.

Trend

The trend (m_t) marks the evolution over time. For example, it could explain the growth in the value of a company's stock.

Seasonality

Seasonality (s_t) is defined as a pattern that repeats with a certain periodicity, which makes it predictable. For example, it could explain the increase in tourists during the high season.

Noise

Noise (Y_t) encompasses all the unknown factors that cannot be explained. For example, it could capture a strike or a disaster.

The additive model of the classical approach would be explained by the following equation:

$$X_t = m_t + s_t + Y_t \tag{2.1}$$

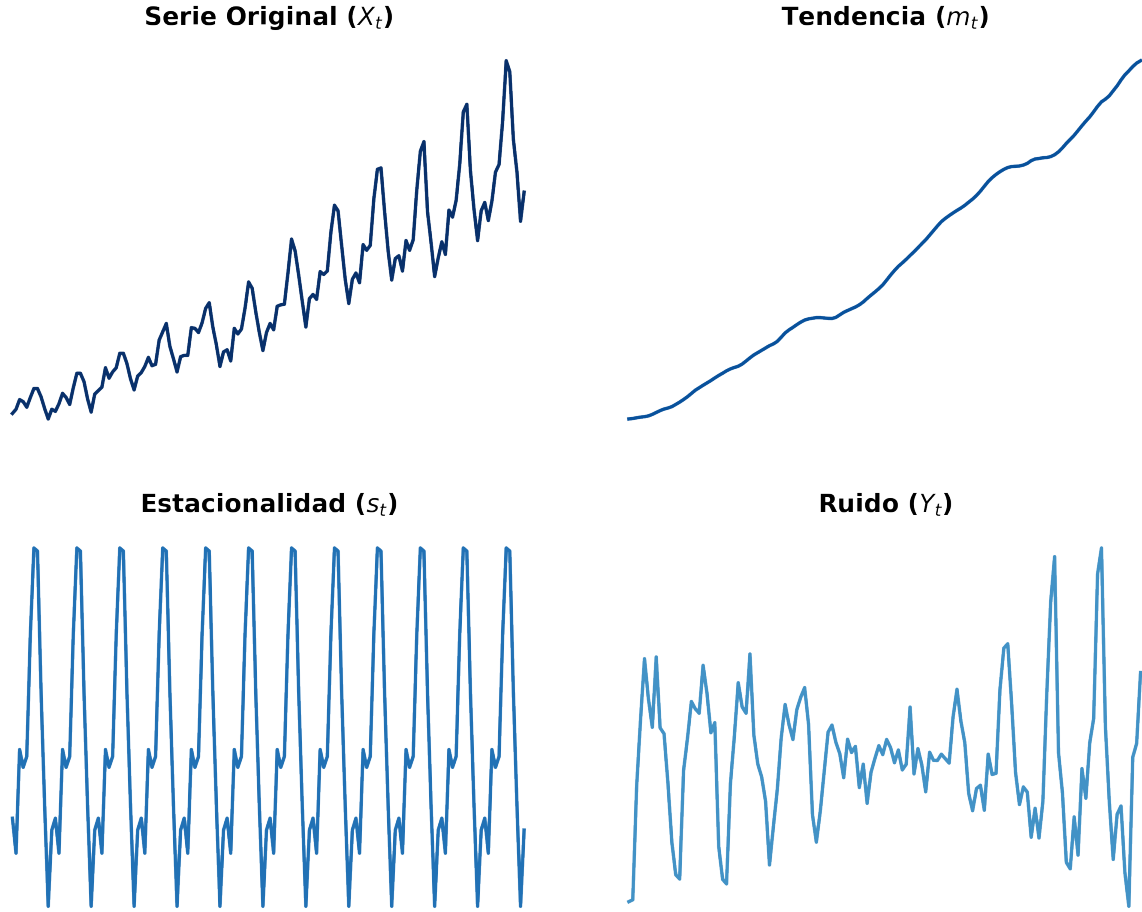


Figure 2.1: Classical decomposition of a time series.

2.2.5 Stationarity

We say that a stochastic process is stationary when its statistical properties (mean, variance, and covariance) remain similar over time (Brockwell & Davis, 2016, p. 13). Therefore, a stationary process shows neither trend nor seasonality, since both of these components vary the statistical properties already mentioned. It should be noted that if a stochastic process is stationary, the time series derived from it will be stationary as well.

Mathematically, a (weakly stationary) stochastic process ¹ satisfies the following properties (Brockwell & Davis, 2016, def. 1.4.2; Box et al., 2008, sec. 2.1.2):

- $E(X_t) = \mu$ is independent of time. That is, $E(X_t) = \mu$ for all t .
- $\text{Cov}(X_t, X_{t+k})$ is independent of time. That is, $\text{Cov}(X_t, X_{t+k}) = \gamma_k$ for $k = \pm_1, \pm_2, \dots$

From these two properties, it follows that the variance also remains constant.

¹There are two types of stationarity: strong and weak. In this thesis, whenever the concept of stationarity is referred to, it will mean weak stationarity.

- $\text{Var}(X_t) = \sigma^2$ for all t .

In Figure 2.2, the graph on the left is stationary, since its variance and mean remain constant. The other graph, however, does not have a fixed mean, since it shows a positive trend.

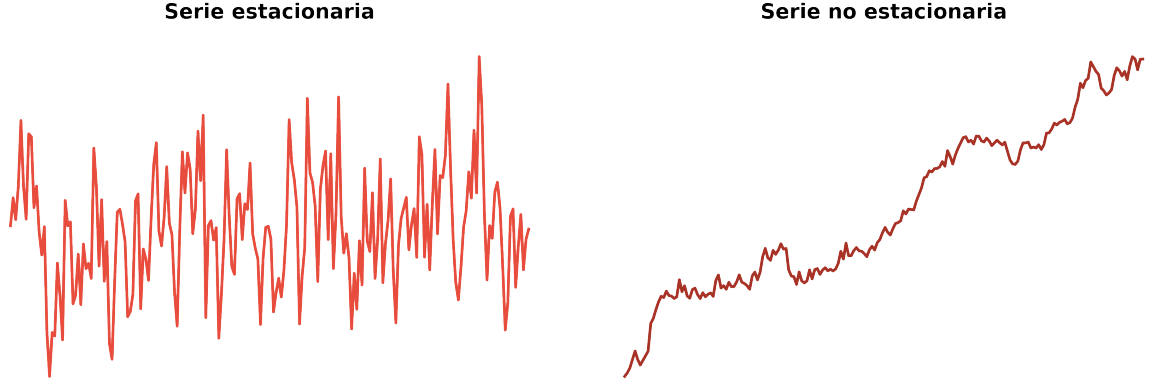


Figure 2.2: Example of a stationary and a non-stationary time series.

2.2.6 Autocorrelation

Lag

The lag is a time interval k that separates one observation x_t from another x_{t+k} .

Autocovariance coefficient

Let X_t be a stationary process; its autocovariance coefficient γ_k at lag k captures all the autocovariances of the process (Box et al., 2008, p. 25).

$$\gamma_k = \text{Cov}(X_t, X_{t+k}) = E[X_t - \mu][X_{t+k} - \mu] \quad (2.2)$$

Autocorrelation coefficient

The autocorrelation coefficient ρ_k measures the degree of linear interdependence between two random variables of the stochastic process separated by a lag k (Box et al., 2008, p. 25).

$$\rho_k = \frac{\gamma_k}{\gamma_0} \quad (2.3)$$

Autocorrelation function (ACF)

The autocorrelation function (ACF) of a stationary process is a function of k that collects all the autocorrelation coefficients ρ_k of the process (Box et al., 2008, p. 29). The plot of this autocorrelation function is called the correlogram.

The correlogram can clearly show whether a time series is stationary.

Partial autocorrelation function (PACF)

While the plot of the simple autocorrelation function (ACF) shows the correlation between two observations and all the intermediate ones, the partial autocorrelation function removes the linear dependence of these intermediate values (Peña, 2010, p. 128).

As with the correlogram of the ACF function, the PACF plot can also be used to determine whether a series is stationary.

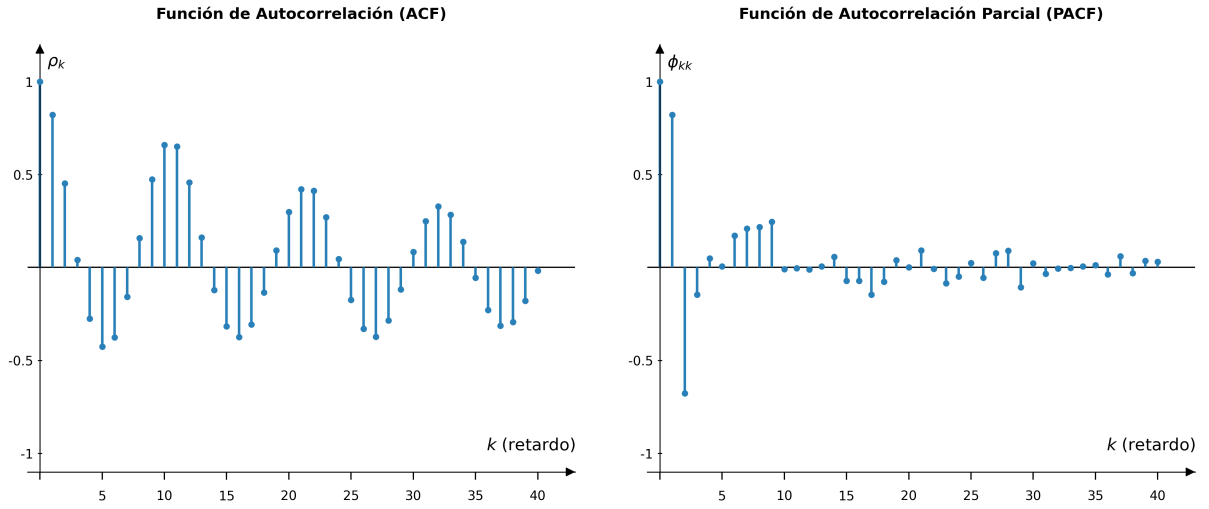


Figure 2.3: ACF and PACF plots of a time series.

2.2.7 The ARIMA family of models

The following presents some of the models from the book (Box et al., 2008) for time series forecasting.

Autoregressive model (AR)

An autoregressive model of order p , denoted $AR(p)$, is a stochastic model in which the current value is determined by past values and by noise.

Assuming without loss of generality that $\mu = 0$, the autoregressive model is explained by the following equation (Shumway & Stoffer, 2025, def. 3.1):

$$x_t = \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + a_t \quad (2.4)$$

If the mean were $E(x_t) = \mu \neq 0$, then we would replace x_t with $x_t - \mu$:

$$x_t - \mu = \phi_1 (x_{t-1} - \mu) + \phi_2 (x_{t-2} - \mu) + \cdots + \phi_p (x_{t-p} - \mu) + a_t \quad (2.5)$$

- The parameters $(\phi_1, \phi_2, \dots, \phi_p)$ make up the vector of autoregressive parameters and are what assign relevance to each past event.
- The term a_t refers to the model's random error. Specifically, it is a random variable that follows a white noise distribution with zero mean and constant variance $a_t \sim \text{WN}(0, \sigma^2)$.

To find an optimal order p , one can refer to the PACF plot (Shumway & Stoffer, 2025, p. 108).

Moving average model (MA)

The moving average model explains each observation x_t using the white noise a_t from the current and past observations (Shumway & Stoffer, 2025, p. 92). Specifically, a moving average model of order q is defined by the following form (Shumway & Stoffer, 2025, def. 3.3):

$$x_t = a_t + \theta_1 a_{t-1} + \theta_2 a_{t-2} + \dots + \theta_q a_{t-q} \quad (2.6)$$

Where the random variables a_t follow a white noise distribution $a_t \sim \text{WN}(0, \sigma^2)$.

For the moving average model, we can rely on the ACF plot to determine an appropriate order q (Shumway & Stoffer, 2025, p. 108).

ARMA

The combination of the two previous models (2.4, 2.6) is called the ARMA model of order p and q .

$$x_t = \mu + \phi_1(x_{t-1} - \mu) + \dots + \phi_p(x_{t-p} - \mu) + a_t + \theta_1 a_{t-1} + \dots + \theta_q a_{t-q} \quad (2.7)$$

The requirement for building this type of model is that the series be stationary (Shumway & Stoffer, 2025, def. 3.5).

ARIMA

The autoregressive integrated moving average model, denoted $\text{ARIMA}(p, d, q)$, is an extension of the $\text{ARMA}(p, q)$ model and is designed to get around this stationarity requirement for series. To do this, the time series is differenced d times until it becomes stationary. The differencing process consists of subtracting the previous observations from the current ones (Shumway & Stoffer, 2025, p. 145):

$$\nabla x_t = x_t - x_{t-1} \quad (2.8)$$

The I in ARIMA stands for Integrated, because to return the data to its original form we must integrate it back d times.

The formula for the ARIMA model is similar to that of the ARMA(p, q) model (2.7)

$$x_t = \mu + \phi_1(x_{t-1} - \mu) + \cdots + \phi_p(x_{t-p} - \mu) + a_t + \theta_1 a_{t-1} + \cdots + \theta_q a_{t-q} \quad (2.9)$$

The difference between these two models is the parameter d already mentioned.

SARIMA

The seasonal autoregressive integrated moving average model (SARIMA) is an extension of ARIMA used when time series exhibit seasonality, that is, when patterns repeat over months or quarters.

It is denoted as SARIMA(p, d, q) $\times$ (P, D, Q) $_m$ (Brockwell & Davis, 2016, p. 117; Hyndman & Athanasopoulos, 2018, sec. 8.9). The parameters p, d, q are the same as in ARIMA; however, the components (P, D, Q) are added, which refer to the seasonal part of the series. The symbol m is the seasonal frequency, that is, the number of periods in each cycle. For example, if the data have daily periodicity and $m = 7$, then the seasonal pattern is weekly. On the other hand, if the periodicity is monthly and $m = 12$, the pattern is annual.

SARIMAX

The SARIMAX model represents the final evolution of this hierarchy by introducing external or exogenous variables (denoted by the letter X). This approach is essential when the time series depends not only on its own past but also on independent external factors such as, for example, public holidays. Mathematically, the SARIMAX model combines a linear regression component for the exogenous variables with error modeling via a SARIMA structure (Hyndman & Athanasopoulos, 2018, ch. 9). This allows the algorithm to not only capture inertia and seasonality but also quantify the impact of specific events.

2.2.8 Box-Jenkins methodology

The Box-Jenkins methodology constitutes an iterative framework for building time series models. Unlike other approaches, it does not assume a model a priori; instead, it uses the data to determine the most suitable structure through a cyclical three-stage process (Box et al., 2008, p. 193):

- 1) **Identification:** In this phase, the stationarity requirement is checked. If the series is not stationary, transformations or differencing are applied (the d component of ARIMA). Afterward, the autocorrelation (ACF) and partial autocorrelation (PACF) functions are used to propose potential orders for the AR and MA components.

- 2) **Estimation:** Once the orders (p, d, q) have been selected, the parameter coefficients are calculated using statistical methods such as maximum likelihood or nonlinear least squares.
- 3) **Diagnostic evaluation:** Whether the model is adequate is checked by analyzing the residuals. For the model to be valid, the residuals must behave like white noise, meaning they must not show significant patterns or autocorrelations. If the diagnosis fails, the process returns to the identification stage to adjust the parameters.

2.2.9 Types of Outliers

Outliers are harmful to statistical analysis, especially for the Box-Jenkins methodology. This is because they artificially increase the variance. As a result, outliers can distort the ACF and PACF plots, leading researchers to make poor decisions. In addition, they introduce bias into the techniques used to estimate the ϕ, θ parameters of the ARIMA models, which considerably impairs their accuracy.

The following describes the taxonomy of anomalous values used within the framework of the ARIMA models.

Additive outlier (AO)

An additive outlier (AO) is considered an isolated error that produces a spike in the series (Chen & Liu, 1993, p. 285). It is observed only at a specific point in time (Box et al., 2008, p. 537), and therefore does not affect future observations.

An example of an AO would be a measurement error in the time series data.

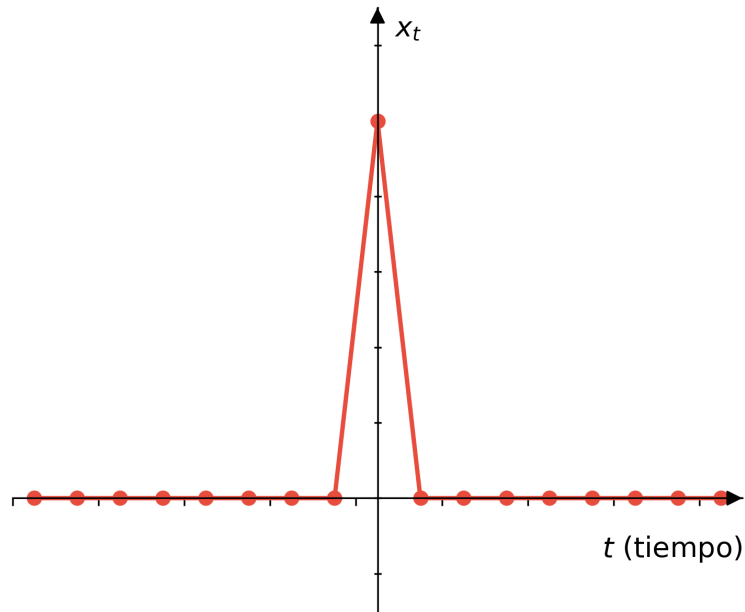


Figure 2.4: Additive outlier.

Innovational outlier (IO)

An innovational outlier (IO) not only affects the series at a specific point in time, but also influences future observations (Box et al., 2008, p. 537). The changes introduced to the series are not a perfect step, but rather depend on the complexity and nature of the process.

An example of this type of outlier could be statements made by an actor. This event can alter box office measurements for the film in which they star.

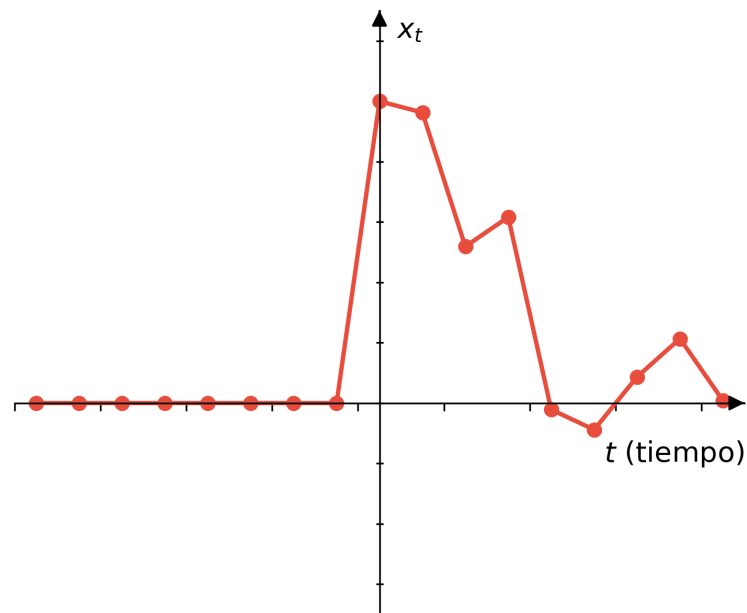


Figure 2.5: Innovational outlier.

Level shift (LS)

A level shift (LS) is an event that abruptly alters the series. It produces a step-like change in level that is permanent (Chen & Liu, 1993, p. 285).

An example of a level shift could be, in the measurement of a race car's speed, the installation of a more powerful engine.

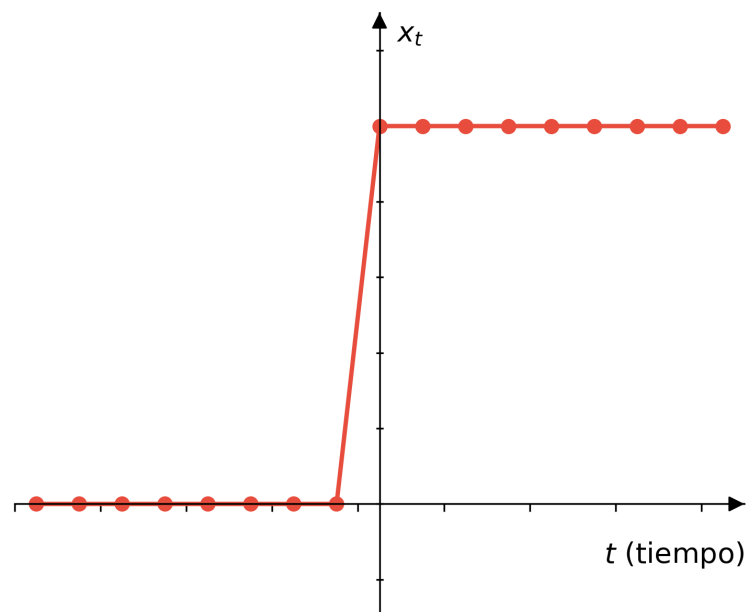


Figure 2.6: Level-shift outlier.

Temporary change (TC)

A temporary change (TC) is similar to a level shift (LS). The difference lies in the fact that the effects of a TC on the series are temporary, while those of an LS are permanent.

This type of extreme value is observed as a sudden change in the data that alters the series, but which decays gradually until the series recovers its original levels (Chen & Liu, 1993, p. 285).

An example of a TC could be a war that affects the price of a barrel of oil, but which, once it ends, returns the price to its previous state.

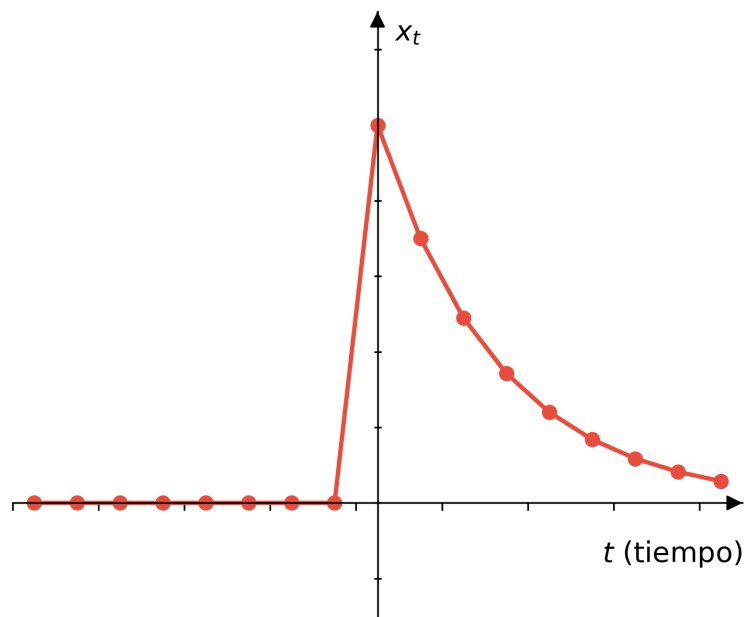


Figure 2.7: Temporary-change outlier.

2.2.10 Iterative procedure for outlier detection

The iterative procedure for outlier detection aims to detect extreme values of the IO and AO types, as well as to effectively estimate the parameters of the ARIMA model in the presence of outliers (Chang et al., 1988, p. 193).

It consists of two stages that are repeated iteratively: one for outlier detection and another for parameter estimation.

Stage 1. Outlier detection

First, the model is fitted to the time series under the assumption that no extreme values exist (Chang et al., 1988, p. 197).

Once this baseline model is available, outlier detection proceeds (Chang et al., 1988, sec. 3.1). To do this, at each instant t of the series, two statistics are computed: one designed to detect IO-type outliers, denoted $\hat{\lambda}_{1,t}$, and another to detect AO outliers, denoted $\hat{\lambda}_{2,t}$.

Next, the largest of all the computed statistics is selected. If the maximum value exceeds a predetermined critical level (usually 3.0, 3.5, or 4.0), we will say that an outlier exists at that instant t , of the corresponding type. Once it has been detected, its impact or magnitude (ω) is estimated.

Finally, the effect of that outlier is removed from the model's residuals and its variance is recalculated. This procedure is repeated iteratively with the newly adjusted errors, ending the process only when no further extreme values are detected.

Stage 2. Parameter estimation

In this stage of the procedure, the model's autoregressive ($\phi_1, \dots, \phi_p$) and moving average ($\theta_1, \dots, \theta_q$) parameters are recalculated (Chang et al., 1988, sec. 3.2). If k outliers were detected in the previous stage, their magnitudes ($\omega_1, \dots, \omega_k$) are estimated together with the model's parameters simultaneously.

With these new parameters, Stage 1 of outlier detection is repeated once again. If no new extreme values are found, the procedure stops for good. Otherwise, Stage 2 continues again.

This algorithm was the first automated method for outlier detection in ARIMA models. However, it had the drawback of detecting outliers individually. This was because, if several consecutive extreme values were found, detecting them one by one caused a “masking effect” (Chen & Liu, 1993, p. 284).

2.2.11 Joint estimation procedure

Five years after the publication of the first method (Chang et al., 1988), the joint estimation procedure was developed (Chen & Liu, 1993).

The method emerges as an evolution of the previous one, with the aim of solving the masking problem caused by the individual detection of extreme values. To do this, the effects of the outliers ($\omega_1, \dots, \omega_k$) are estimated jointly using a regression model (Chang et al., 1988, p. 287). In addition, two new types of outliers are proposed: the LS and TC types (Chen & Liu, 1993, p. 285).

2.2.12 Machine Learning

Machine learning refers to the techniques by which computer systems learn from data (Goodfellow et al., 2016, p. 8). Instead of a human explicitly programming a task, the computer itself works out the solution.

Machine learning algorithms are used in a multitude of scenarios, such as recommendation algorithms, business decision-making, or computer vision.

2.2.13 Neural Network

A neural network is a machine learning computational model inspired by the functioning of the human brain itself (Aggarwal, 2017, p. 98). The network is made up of neurons organized into layers. Each neuron receives information from the neurons in the previous layer through connections that have an associated weight. The neuron computes the weighted sum of these inputs and applies a nonlinear activation function to it. The first layer of the network is known as the input layer, the following layers as hidden layers, and the final one as the output layer.

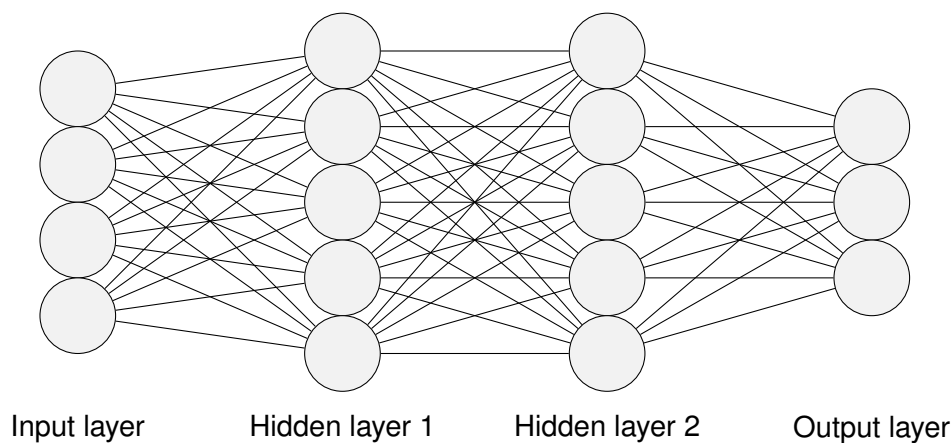


Figure 2.8: Example of a neural network architecture.

For the network to be effective, suitable weights must be found. To do this, the neural network is trained, first by calculating the prediction error with a loss function and then correcting it using mathematical algorithms such as gradient descent and backpropagation.

2.2.14 Activation function

An activation function is a mathematical operation, generally nonlinear, that is applied to the outputs of the neurons in neural networks (Goodfellow et al., 2016, p. 172). The purpose of these functions is to introduce nonlinearity into the model: if the network remains completely linear, using more layers will be redundant, and it will also be unable to learn complex behaviors (Aggarwal, 2023, sec. 1.5). There are several types of activation functions; some of the most common ones are explained below.

ReLU

This is the most recommended activation function, and the one used by default for the hidden layers of the network. The Rectified Linear Unit (ReLU) is a piecewise-defined function that returns zero if

the input is negative, and returns the same value as the input if it is positive:

$$f(x) = \max(0, x) \quad (2.10)$$

Its popularity is due to the fact that it is extremely computationally efficient and greatly mitigates the vanishing gradient problem, which makes it possible to train deep neural networks more quickly.

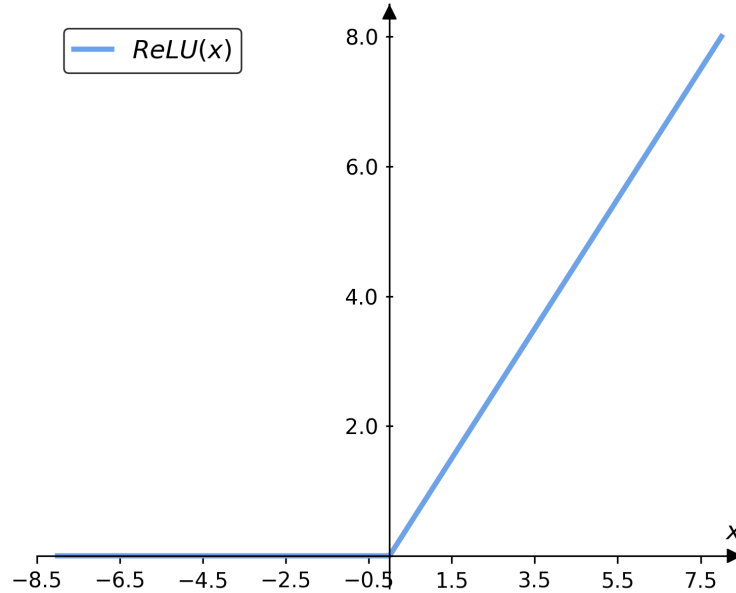


Figure 2.9: ReLU function.

Sigmoid

The sigmoid function transforms any real value into a value within the interval $[0, 1]$. Its mathematical expression is as follows:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.11)$$

This function is most commonly used in the output layer. Since it returns values between 0 and 1, it can be interpreted as the probability and confidence of the prediction.

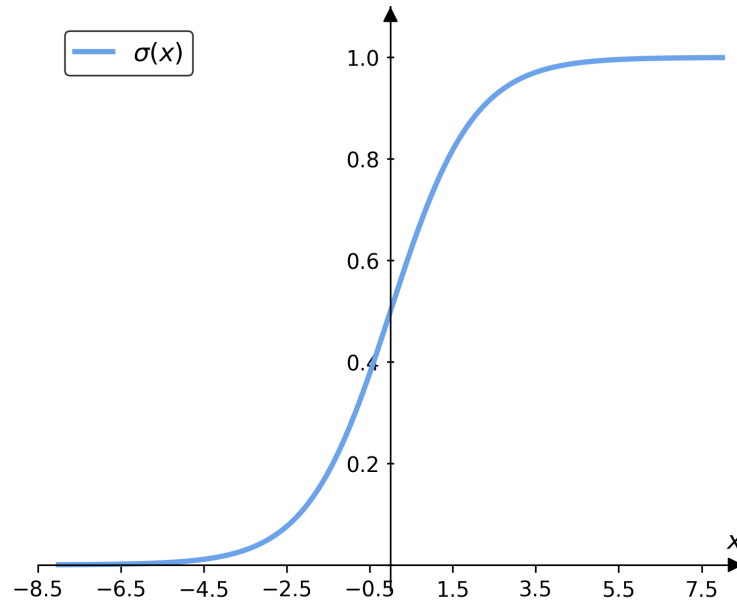


Figure 2.10: Sigmoid function.

Hyperbolic tangent

The hyperbolic tangent function is a rescaled and shifted version of the sigmoid function. Unlike the sigmoid, the hyperbolic tangent can return negative values. This is because its range now lies between $[-1, 1]$.

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2.12)$$

Being centered at zero facilitates the convergence of the optimization algorithm during training. Although it performs better than the sigmoid function in the hidden layers, it still suffers from the vanishing gradient problem at the extremes of the function.

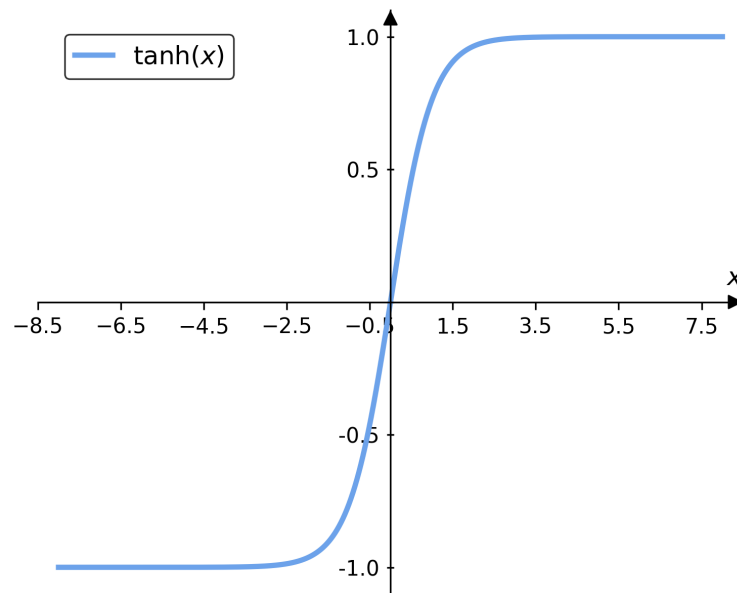


Figure 2.11: Hyperbolic tangent function.

2.2.15 Recurrent Neural Networks (RNN)

Recurrent neural networks are a type of neural network specialized in processing sequential data (Goodfellow et al., 2016, p. 373). For this reason, they are used for natural language processing, speech recognition, or for forecasting time series.

This type of network uses feedback connections to maintain a “memory” over time. However, recurrent neural networks suffer from the vanishing and exploding gradient problem. When the gradient persists over long periods of time, it tends to vanish or explode (Goodfellow et al., 2016, p. 401). This complicates learning long-range interactions in time.

2.2.16 LSTM

LSTMs (Hochreiter & Schmidhuber, 1997) are a type of RNN that addresses the vanishing gradient problem² by introducing memory cells. The cells split the flow of information into an internal state (C_t) and a hidden state (H_t). This architecture has two goals: on the one hand, to protect long-term memory by allowing the internal state to be updated through additive linear operations (thereby mitigating vanishing); and on the other, to use the hidden state to filter that memory and expose only the relevant information at each time step.

²In contrast, to tackle the exploding gradient problem, the gradient clipping technique is commonly used during training (Goodfellow et al., 2016, sec. 10.11.2).

How memory cells work

Memory cells are built using a system of multiplicative gates that control which information enters, leaves, or is forgotten. In addition, they regulate how memory is used for the output.

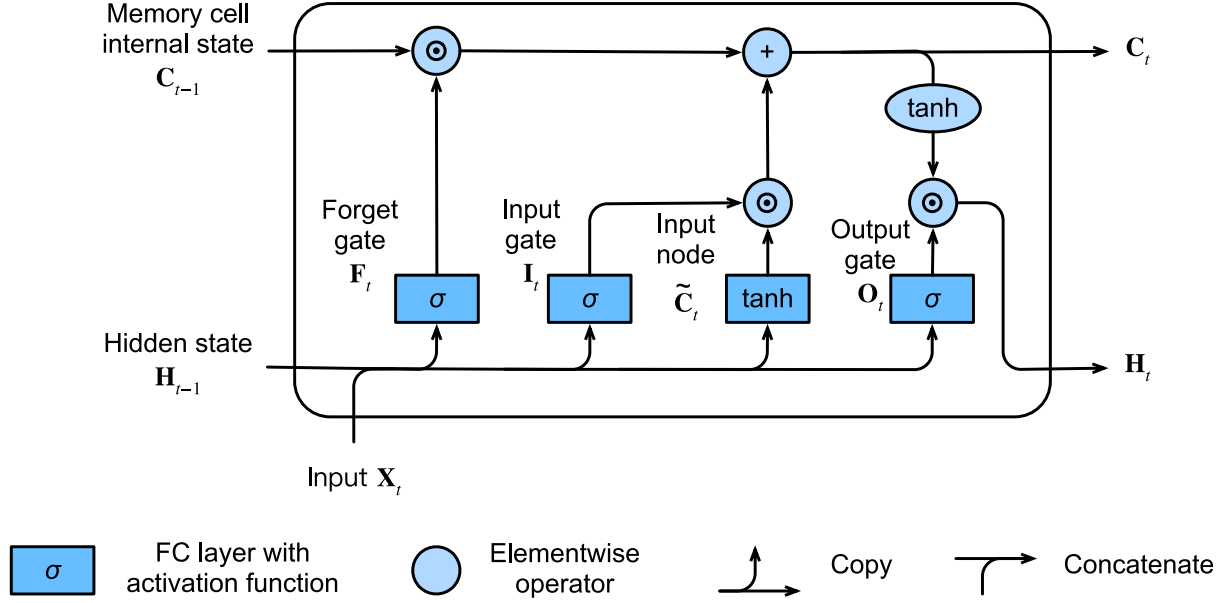


Figure 2.12: Diagram of an LSTM cell. Source: Zhang et al., 2023.

Figure 2.12 illustrates the internal workings of a memory cell. The top line shows the flow of the cell's internal state (C_t), which is interpreted as long-term memory. The bottom line, on the other hand, follows the cell's hidden state (H_t), which constitutes short-term memory. Besides these two arrows, we see element-wise operations, activation functions, and the gates discussed above.

The operation of the memory cell is explained in three steps:

- 1) The forget gate F_t decides what information from long-term memory to retain.
- 2) The input gate I_t decides what information to add to long-term memory.
- 3) The output gate O_t decides what information from long-term memory is relevant in the short term H_t .

First, on the bottom line, the short-term memory from the previous cell H_{t-1} is received, to which new information X_t is concatenated. The first gate this concatenated vector $[H_{t-1}, X_t]$ passes through is the forget gate F_t . In doing so, the vector is multiplied by a weight matrix W_f and a bias vector b_f is added to it. Finally, the previous calculation is passed through a sigmoid activation function, which produces a vector whose values lie between 0 and 1.

$$F_t = \sigma(W_f \cdot [H_{t-1}, X_t] + b_f) \quad (2.13)$$

This vector is multiplied element-wise with the long-term memory of the previous cell $\mathbf{C}_{t-1}$, which reduces its impact, producing the forgetting effect. This is because the vector's components are values between 0 and 1 that act as the percentage of information retained from long-term memory.

$$\mathbf{F}_t \odot \mathbf{C}_{t-1} \quad (2.14)$$

The second gate corresponds to the input gate $\mathbf{I}_t$. This determines what part of a candidate memory $\tilde{\mathbf{C}}_t$ will be added to the cell's internal state. Mathematically, it is modeled in the same way as the previous gate (2.13).

$$\mathbf{I}_t = \sigma(W_i \cdot [\mathbf{H}_{t-1}, \mathbf{X}_t] + b_i) \quad (2.15)$$

The candidate memory $\tilde{\mathbf{C}}_t$ is known as the input node and is calculated taking into account the input information $\mathbf{X}_t$ and the previous hidden state $\mathbf{H}_{t-1}$. The mathematical operations are similar to those of the other gates (2.13, 2.15); however, the activation function is changed to the hyperbolic tangent. The reason for using the hyperbolic tangent is that this function returns values between -1 and 1, which allows the internal state $\mathbf{C}_t$ to take on negative values and invert information if necessary.

$$\tilde{\mathbf{C}}_t = \tanh(W_c \cdot [\mathbf{H}_{t-1}, \mathbf{X}_t] + b_c) \quad (2.16)$$

In short, the long-term memory $\mathbf{C}_t$ is computed as follows: the forget vector $\mathbf{F}_t$ is computed and multiplied element-wise with the previous state $\mathbf{C}_{t-1}$ to decide what percentage of information to remember. Next, the input node $\tilde{\mathbf{C}}_t$ is determined as a candidate piece of information to incorporate. The percentage of this information that is ultimately added is determined by the input gate vector $\mathbf{I}_t$, multiplying both element-wise. Finally, this product is added to the long-term memory through an element-wise sum.

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t \quad (2.17)$$

The last gate to discuss is the output gate $\mathbf{O}_t$. This gate regulates what percentage of long-term memory will form the cell's hidden state $\mathbf{H}_t$.

The hidden state not only influences long-term memory but is also used to make predictions at that exact instant t . Therefore, the output gate has two interpretations: it determines what part of the current cell state's information to send to the next time step, and it also regulates what part of long-term memory is relevant at the current moment.

$$\mathbf{O}_t = \sigma(W_o \cdot [\mathbf{H}_{t-1}, \mathbf{X}_t] + b_o) \quad (2.18)$$

The cell's short-term memory $\mathbf{H}_t$ is obtained by applying the element-wise multiplication of the output gate vector $\mathbf{O}_t$ with the hyperbolic tangent of the internal state $\mathbf{C}_t$.

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) \quad (2.19)$$

2.2.17 Outlier detection with LSTM

The procedure for detecting outliers using LSTM networks consists of modeling the system's typical behavior in order to later identify anomalies.

Specifically, the following steps are followed (Malhotra et al., 2015, p. 90):

- 1) First, the LSTM network is trained with data free of anomalies. This is one of the biggest disadvantages of using LSTM for outlier detection, since it is the researchers who must manually screen the data.
- 2) Once the model is trained, several predictions are generated for each instant t of the series. These predictions are generated from different points back in time ($t - 1, t - 2, \dots$).
- 3) For each instant t , an error vector is constructed by comparing the predictions with the actual observations.
- 4) Finally, on an anomaly-free validation subset, a multivariate Gaussian distribution is fitted to the error vectors, thereby characterizing the error behavior on anomaly-free data.

Once the parameters of the Gaussian distribution are known, the researchers can calculate the probability of whether an observation is anomalous or not.

2.2.18 Convolutional Neural Networks (CNN)

Convolutional neural networks are a type of neural network specialized in processing data in a grid-like format. The dimensionality of this grid can vary depending on the data format, for example, one dimension for time series and two dimensions for images. The name convolutional network comes from the mathematical operation it uses, convolution (Goodfellow et al., 2016, p. 330).

Advantages of convolution

There are several reasons for using convolutions.

- 1) First, the use of convolutions makes it possible to reduce the model's computational and memory cost. If we used a conventional neural network to process a 1024×1024 pixel image, we would require 2^{20} weights in each of its neurons. In a layer of just 2048 neurons, we would already have 2^{31} parameters. The problem lies in taking every individual pixel for the output, since the local neighborhood of each pixel is usually redundant. Convolution makes it possible to take local regions, which significantly reduces the number of parameters in the model (Goodfellow et al., 2016, p. 335).

- 2) Another major advantage of convolution is that it enables parameter sharing. Instead of learning a set of weights for each specific region of the image, kernels are learned that slide across and are used at every position of the input. This drastically reduces the parameters and improves the model's computational efficiency (Goodfellow et al., 2016, pp. 335–338).
- 3) Thanks to the previous property, the model has “equivariance to translation”: if the input shifts, the output will change in exactly the same way (Goodfellow et al., 2016, pp. 338–339). In time series, this means that the detected patterns are invariant over time, again reducing the neural network's computations.
- 4) Finally, convolution makes it possible to work with variable-sized input data (Goodfellow et al., 2016, p. 335). This is practical in production, since the model can be trained on one section of the series and then used on massive data sequences.

One-dimensional convolution

In the particular case of this thesis, the data have a single dimension (time). In addition, since they consist of observations x_t at specific instants t , we say they are discrete data. Therefore, discrete one-dimensional convolutions will be used, mathematically described by the following operation³:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t - a) \quad (2.20)$$

Where:

- x represents the observations of the time series.
- w is the kernel. It consists of a vector of weights.
- s is the result of the operation. It is called the feature map.

The convolution process consists of “sliding” the kernel along the sequence. For each position t , the “neighbors” are multiplied by the elements of the kernel and all the results are summed. The output $s(t)$ can be interpreted as a weighted average at instant t .

Architecture of a CNN

We can divide the network's architecture into two parts: feature extraction and classification or prediction with dense layers.

³In mathematics, pure convolution requires the kernel to be flipped before sliding across the data series. Because of this, most machine learning libraries use cross-correlation instead of convolution; however, the community still calls it convolution.

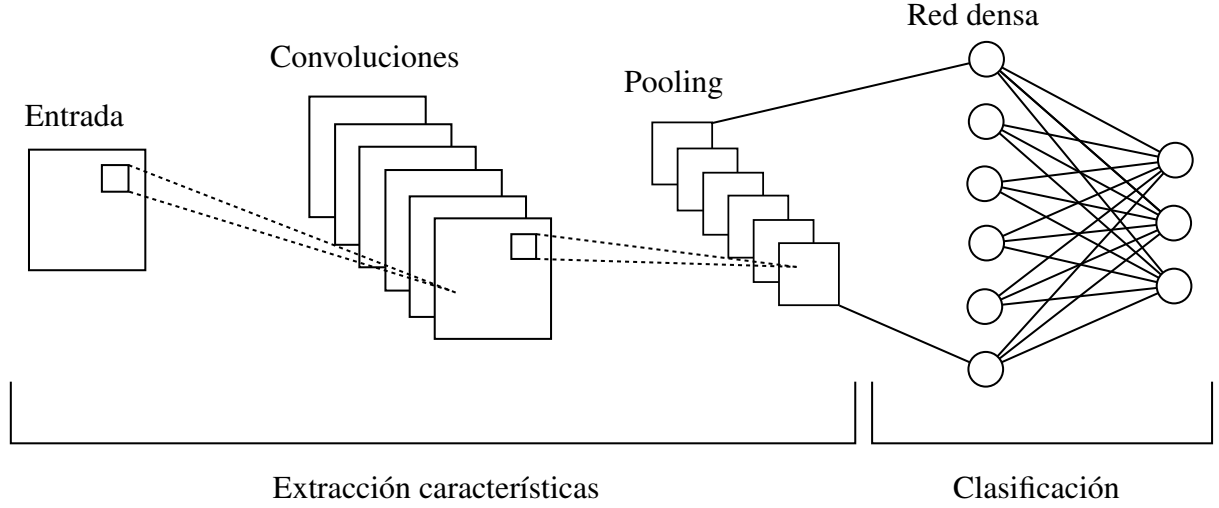


Figure 2.13: Architecture of a CNN.

In the feature extraction stage, a block made up of three specialized layers is used: convolution, pooling, and ReLU (Aggarwal, 2023, p. 308). This block can be repeated several times to create a deeper network.

The first part of the block consists of a convolutional layer made up of one or more kernels, which are called filters. The number of filters or channels in the convolutional layer is called depth (Aggarwal, 2023, p. 307). Each filter has different weights and is therefore said to specialize in a particular pattern in the data. In images, for example, it could focus on detecting people's eyes.

The output of each filter is a feature map. To introduce nonlinearity, an activation function is applied to each cell of the feature map, the most common being ReLU (Goodfellow et al., 2016, p. 339).

Finally, a pooling layer is used to reduce the dimensionality of the data. This eases the computational load and also makes the network more resilient to translations, meaning the results do not change when the image is moved. There are several pooling functions, one of them being Max Pooling. This function takes a rectangular region, for example 3×3 , and slides it across the data. For each region covered by the window, the function returns the maximum value. Depending on the stride we choose (how many pixels the window moves), the data will be reduced, with the new dimensions calculated using Equation 2.21 (Aggarwal, 2023, p. 315).

$$\begin{aligned} L_{nueva} &= \left\lfloor \frac{L_q - P_q}{S_q} \right\rfloor + 1 \\ B_{nueva} &= \left\lfloor \frac{B_q - P_q}{S_q} \right\rfloor + 1 \end{aligned} \quad (2.21)$$

Where:

- L_q : is the length of the input matrix or feature map.

- B_q : is the breadth of the input feature map.
- P_q : is the spatial size of the pooling window (the filter).
- S_q : is the stride with which the window slides.
- $\lfloor \dots \rfloor$: indicates the mathematical truncation operation (rounding down to the nearest integer).

For example, with a 50×50 matrix and a stride of 3 (no overlap), the dimension is reduced to 16×16 . With a stride of 2, to 24×24 , and with a stride of 1, to 48×48 .

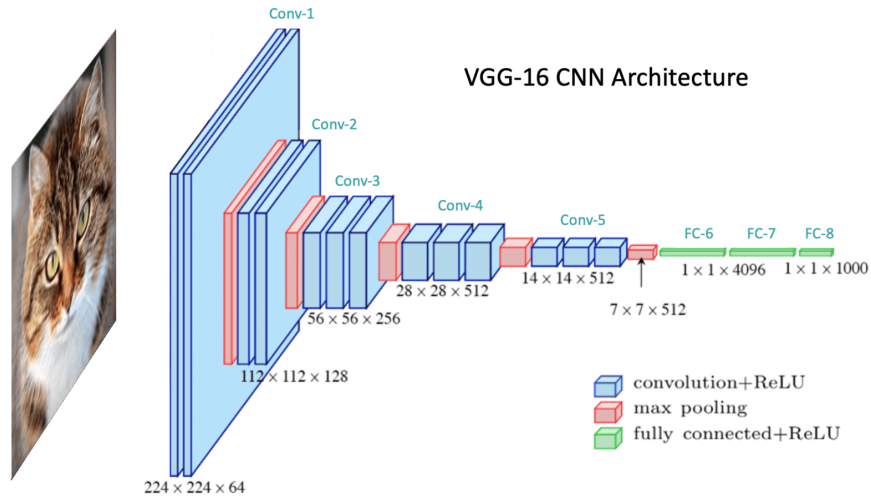


Figure 2.14: Architecture of VGG-16. Source: github.com/smallick/learnopencv.

As can be seen in Figure 2.14, a common practice in deep CNNs is to halve the dimensionality of the data with the pooling function, but double the number of filters in the next block.

Once feature extraction is complete, the classification or prediction stage follows. In the case of two-dimensional data such as images, before classifying, the feature map data must be flattened with a flatten layer. This is because traditional neural networks take a one-dimensional vector as input. The data is then classified or predicted (depending on the task) using a fully connected or dense neural network like the one in Figure 2.8.

2.2.19 DeepAnT

DeepAnT is a methodology for outlier detection in time series based on convolutional networks. According to its researchers, this algorithm outperforms other techniques developed by companies such as Yahoo, Twitter, and Numenta. To support this claim, the authors evaluated 15 state-of-the-art methods on 10 datasets (totaling 433 time series), with DeepAnT achieving the best results in

most cases (Munir et al., 2019, p. 1992). It is also notable for being an unsupervised algorithm, which makes it possible to train the model without having to previously remove or label anomalous observations; a feature that makes it well suited to real production environments. Finally, thanks to the parameter sharing inherent to CNN architectures, DeepAnT has the additional advantage of not requiring large volumes of historical data for training (Munir et al., 2019, abstract).

DeepAnT's architecture is split into two modules (Munir et al., 2019, p. 1994). The first module (*Time Series Predictor*) predicts the next values of the time series using convolutional networks. The second module (*Anomaly Detector*) is responsible for labeling a point as anomalous using the predictor.

Predictor module

The prediction module consists of a CNN with two convolutional layers, two max pooling layers, and one fully connected layer.

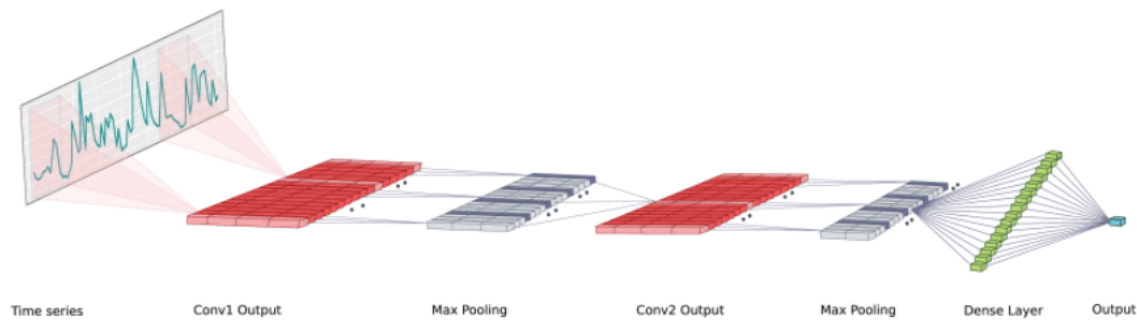


Figure 2.15: Architecture of DeepAnT. Source: Munir et al., 2019.

As already mentioned, training begins with feature extraction. For this, the following configuration is used (Munir et al., 2019, p. 1996):

- Convolutional layers with 32 filters of size 3×3 .
- ReLU activation function.
- Max Pooling of size 2×2 .
- MAE loss function.
- SGD optimizer.

One detail of training CNNs on time series is the use of the “historical window” (Munir et al., 2019, p. 1995). Instead of training the model with all the data, it is “chunked” into windows of size w . Specifically, when the network is going to predict a value x_t , the values $\{x_{t-w}, \dots, x_{t-2}, x_{t-1}\}$ are taken as context. The size of the historical window w is considered a new hyperparameter of the model that affects the quality of the predictions.

Detector module

The categorization of observations is carried out using the predictions from the previous module. The procedure consists of giving each observation an anomaly score using the Euclidean distance formula (Munir et al., 2019, p. 1996):

$$(y_t, y'_t) = \sqrt{(y_t - y'_t)^2} \quad (2.22)$$

Where:

- (y_t, y'_t) is the anomaly score.
- y_t is the observation to be classified.
- y'_t is the prediction generated by the neural network.

To determine whether an observation is an outlier or not, this score is computed and compared against a threshold. If the distance exceeds this threshold, the point is labeled as an outlier. This threshold is defined by the researchers depending on the particular case.

2.3 Related work

This section presents work related to outlier detection in the field of network traffic. First, a comparison is presented between a classical statistical model and one based on machine learning. Subsequently, various studies focused on developing hybrid models that integrate the techniques discussed previously in this thesis are described.

2.3.1 A Comparison of ARIMA and LSTM in Forecasting Time Series

In this work, Siامي-Namini et al. (2018) carry out a comparative analysis between the traditional statistical model ARIMA and the long short-term memory (LSTM) neural network for time series forecasting. To validate their performance, the authors evaluated both models using 12 datasets with monthly periodicity, whose records span the period from 1985 to 2018.

In the study, they concluded that deep learning-based models were superior to traditional algorithms. Specifically, they observed an average improvement of 85% in LSTM predictions compared to ARIMA.

Although the analysis is oriented toward time series forecasting, this thesis has already explained the anomaly detection process based on the prediction errors of the LSTM and ARIMA models.

2.3.2 Web traffic anomaly detection using C-LSTM neural networks

In this work, Kim and Cho (2018) present a methodology for detecting anomalies in web traffic through the development of a hybrid model called C-LSTM. This architecture integrates a CNN for spatial feature extraction and an LSTM for modeling temporal dependencies.

The model's performance was evaluated against various machine learning techniques using Yahoo's *Webscope S5* dataset and applying 10-fold cross-validation. The C-LSTM model obtained the best results, outperforming conventional algorithms such as *Isolation Forest* and KNN. Likewise, the research showed that the hybrid combination significantly improves precision compared to using the CNN and LSTM architectures in isolation.

Finally, the authors added a deep neural network (DNN) to optimize the separability of the data. With the complete architecture (CNN + LSTM + DNN), the researchers reported an accuracy of 98.6% using the same dataset.

2.3.3 Time-Series Anomaly Detection Service at Microsoft

In this article, Ren et al. (2019) develop an anomaly detection system for time series for Microsoft's services. The system monitors the most relevant business metrics, such as revenue or views, in real time.

The model uses an adapted spectral residual (SR) model, a technique traditionally used in the field of computer vision to detect the most salient areas of an image. In the work, the model was used to detect the most salient parts of the data. Then, a CNN decided whether the observation was anomalous or not.

One of the main advantages of this SR-CNN architecture is its fully unsupervised nature, an indispensable feature when working with the data volumes of companies like Microsoft. After being evaluated in environments with real production data, the hybrid model demonstrated not only high computational efficiency, but also a strong capacity for generalization across various types of time series.

Chapter 3

Methodology

This chapter justifies the methodological and technical decisions adopted during the development of the thesis. First, the methodology followed is described, detailing the phases of the project, the models chosen to address anomaly detection, and the criteria that motivated their selection. Next, the technologies and libraries used for the implementation are listed, together with the computational resources used during the training and evaluation of the models.

3.1 Methodology

To structure the development of this thesis and meet the stated objectives, a sequential methodology divided into the following phases has been defined.

3.1.1 Problem research

This first phase focused on the theoretical exploration and literature review concerning anomaly detection methodologies in time series. Through the analysis of existing techniques, a solid frame of reference was established, which made it possible to justify and define the choice of the most relevant predictive models to address the particularities of the use case.

3.1.2 Exploratory analysis and data preprocessing

Once the dataset was loaded, a brief preprocessing step was carried out in order to be able to handle it properly.

An exploratory data analysis was then carried out. To this end, various plots were generated in order to visually identify underlying patterns, such as seasonality, cycles, or trends.

3.1.3 Manual labeling of incidents

In order to objectively evaluate the detection models, it was necessary to have a set of labeled incidents available. Since the dataset did not include such labels, they were marked manually.

3.1.4 Design and implementation of predictive models

Once the labeling was completed, the models were selected and trained. As already explained above, the detection of extreme values will be based on the forecasts produced by these models.

In this phase, methodologies of different mathematical natures were implemented to cover various learning approaches. As a fundamental criterion for their selection, it was established that the models had to allow the integration of exogenous variables. This capability proved essential in order to incorporate information corresponding to public holidays, whose impact alters the usual behavior of the time series.

Additionally, before addressing the more sophisticated models, a baseline reference model was included in order to compare the degree of improvement provided by the other techniques.

Specifically, the following models were selected:

Baseline model

A detector inspired by the one already used by the company was implemented. It is based on the average of the value observed at the same time during the five preceding equivalent days, complemented by a set of rules that distinguish between total and partial outages. The simplicity of the detector made it possible to objectively evaluate the degree of improvement achieved by the more complex algorithms.

SARIMAX

The inclusion of a model belonging to the ARIMA family is justified by its historical relevance and its standing as the classic methodological standard in time series forecasting. The SARIMAX variant was chosen in order to capture the seasonal patterns inherent to the data traffic and, in addition, to add information on public holidays by means of exogenous variables.

LSTM

This recurrent neural network architecture was selected due to its capacity to process sequential data and capture long-term temporal dependencies, a feature that is essential for modeling the complex patterns present in network traffic. Moreover, as already explained in the state of the art,

the literature provides evidence that deep learning models outperform traditional algorithms such as ARIMA (Siame-Namini et al., 2018). Additionally, this implementation also allows the integration of exogenous variables.

CNN

Finally, a one-dimensional convolutional network was incorporated. The choice of this model is based on two characteristics that are particularly well suited to this specific thesis.

First, its high computational efficiency stands out; CNNs are significantly faster than recurrent networks in the training and prediction phases. This processing speed is a major advantage for the deployment of the real-time anomaly detection system proposed in this thesis.

Second, as evidenced in the literature through architectures such as *DeepAnT* (Munir et al., 2019), convolutional networks require smaller volumes of data during the training phase, maintaining high performance without the need for massive datasets.

3.1.5 Hyperparameter tuning

Once the models were implemented, their hyperparameters were optimized. For this purpose, an automated search procedure was used, which evaluates different configurations and selects the one that maximizes performance on the validation set.

3.1.6 Evaluation and comparative analysis

Next, the different models were evaluated and compared according to their ability to detect the previously labeled incidents. Two different metrics were used to quantify performance. The first is a custom event-based metric, designed specifically for this use case. The second is the NAB score (Ahmad et al., 2017), which is widely established in the time series anomaly detection literature.

3.1.7 Deployment for real-time anomaly detection

Finally, a platform for real-time anomaly detection and visualization was developed. The goal of this last stage is to prototype a possible real-world solution to the company's problem.

3.2 Technologies used

3.2.1 Python 3.11.8

Python was selected as the programming language for this thesis. This decision is based on its versatility, its development speed, and, above all, the wide ecosystem of libraries available for data analysis and machine learning.

The Python libraries used in the development are the following:

NumPy 2.4.4

A library that underlies several data analysis libraries. It has been used to perform mathematical transformations on the data.

Pandas 3.0.3

Pandas is the main library for the manipulation and analysis of structured data. In this thesis it has been used to load the dataset and to preprocess it.

Matplotlib 3.10.9

The reference module for generating plots in Python. It has been used in the exploratory data analysis (EDA) and in creating the figures included in this document.

Plotly 6.7.0

An interactive visualization library for Python. In this thesis it has been used to build the viewer with which the incidents in the dataset were manually labeled.

Statsmodels 0.14.6

A library specialized in statistical analysis within the Python ecosystem. In this research, it has been used to run hypothesis tests and to build and fit the ARIMA predictive models.

Statsforecast 2.0.2

A library specialized in classic time series models. In this thesis it has been used for the automatic fitting of the SARIMAX model by means of its `AutoARIMA` algorithm.

Scikit-learn 1.8.0

The reference library for general machine learning tasks in Python. It has been used to scale the input variables of the neural networks by means of `MinMaxScaler`.

Keras 3.14.1

TensorFlow is a fundamental open-source library aimed at building machine learning models. Keras acts as its high-level Application Programming Interface (API), making it easier to use. In this thesis, it has been used to assemble and train the convolutional (CNN) and recurrent (LSTM) neural network architectures used for anomaly detection.

Optuna 4.8.0

A hyperparameter optimization library using automated search. It has been used to find the configuration that maximizes the detection F_1 of the DeepAnT and LSTM models, storing the results in a SQLite database.

Holidays 0.96

A library that compiles the official public holidays of a large number of countries. It has been used to build the exogenous variable `festivo` from the Spanish national calendar.

Dash 4.1.0

A lightweight web application framework in Python, built on top of Plotly. It has been used to implement the dashboard of the real-time anomaly detection application.

3.2.2 Jupyter Notebook

The code was developed in Jupyter notebooks. This tool makes it possible to document the project using Markdown cells. It also speeds up development by avoiding the need to run the entire project every time a change is made. It achieves this by separating the code into independent cells that can be run individually. Another advantage of this environment is that it saves the execution results; this makes it possible to share the document without the recipient having to run it again.

3.2.3 Computational resources

The training of the proposed models was carried out locally. Specifically, the physical machine has an NVIDIA GeForce RTX 2070 graphics card. The use of this dedicated hardware has proved essential

to significantly speed up the training process of the deep neural networks.

Chapter 4

Development

This chapter presents the development of the thesis. First, the preprocessing of the dataset and the manual labeling of the incidents are documented, followed by the exploratory analysis of the traffic series. Next, the common evaluation framework is defined and the four proposed detectors are implemented. The chapter concludes with the hyperparameter tuning, the comparison of results, and the description of the real-time monitoring dashboard.

4.1 Data preparation and preprocessing

Real production data from 2023, 2024, and 2025 were used, from an application that provides service at the national level. The partner company for this thesis wishes to remain anonymous, so details about this application will be omitted. The only thing worth mentioning is that it has servers in several Spanish provinces.

Table 4.1: Dataset structure.

Attribute	Data type	Description
Operation	String	Request type (query or download).
Date-time	Timestamp	Exact date and time at which the operation is recorded (format YYYY-MM-DD HH:MM).
Metric	Integer	Number of requests per minute received by the application.

As can be seen in Table 4.1, each observation has 4 columns. The *Operation* column indicates whether the request was a query or a download. For this research, only query-type requests will be used, since they are more abundant.

The *Metric* attribute contains the number of requests received at the national level in the last minute. Despite having records every minute, for the purposes of this thesis the time series was resampled

to a 5-minute periodicity. The reason for this resizing is to speed up model training and ease the memory load on the machines. In addition, better performance was observed than with a one-minute resample, possibly because it smoothed out the noise in the series.

The original dataset was split across 1096 CSV files. The first operation consisted of concatenating all their rows into a single Pandas dataframe. The *Date-time* field was then processed so that it had the correct variable type. In addition, the series was filtered to work only with queries.

Figure 4.1 shows the resulting time series once the previous operations have been applied. The pronounced daily and weekly seasonality of the traffic can be observed, as well as an increase in request volume in October 2025.

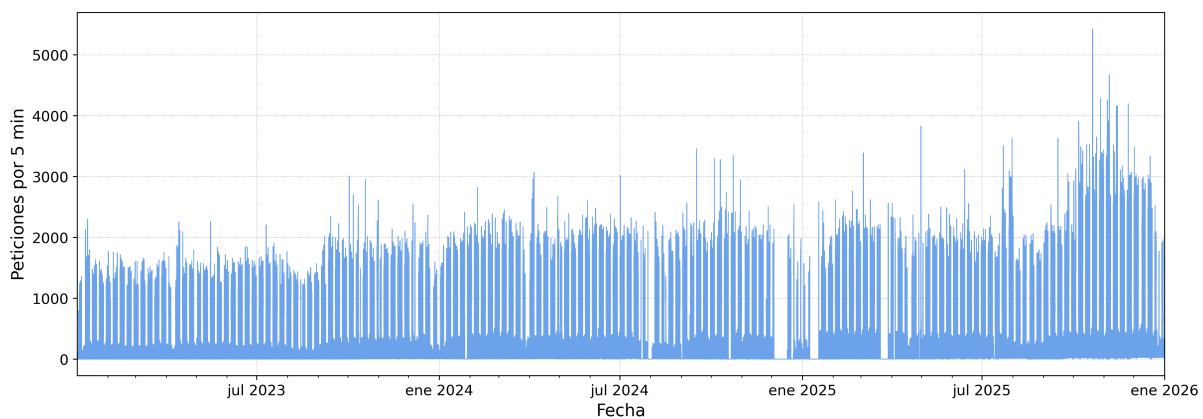


Figure 4.1: Time series of the data.

4.1.1 Labeling of outliers

It is essential to know the anomalies in the data in order to build and validate the detection models. However, since this is real-world data, the observations were not labeled. The only record obtained was a messaging conversation in which server incidents were reported. From this chat, a structured table was obtained using regular expressions.

Table 4.2: Structure of the processed incidents.

Attribute	Data type	Description
Inicio	Timestamp	Start date and time of the event (YYYY-MM-DD HH:MM:SS).
Fin	Timestamp	End date and time of the event.
Duracion_horas	Decimal	Total elapsed time expressed in hours (e.g., 1.50).
Titulo	String	Title of the incident.
Descripcion	String	Extended detail of the problem.
Num_mensajes	Integer	Total number of messages associated with the incident.
Estado	String	Current status of the incident.
Autores	String	Author of the message.

In total, 21 anomalies associated with application outages were recorded for the period 2023–2025.

4.1.2 Manual review of outages

Despite having a record of these 21 incidents, review of the time series revealed that numerous service outages had not been recorded. Therefore, an exhaustive manual screening of the data had to be carried out.

During this process, interruptions were classified into two categories: total and partial. A total outage was considered to be one in which the flow of requests dropped to zero, whereas partial outages were defined as those showing a drastic reduction in traffic without reaching a complete interruption.

Figure 4.2 illustrates both behaviors on the real query series: in the upper case, traffic abruptly drops to zero during the incident interval, while in the lower case a degradation in request volume can be seen that, nevertheless, does not drop to zero completely.

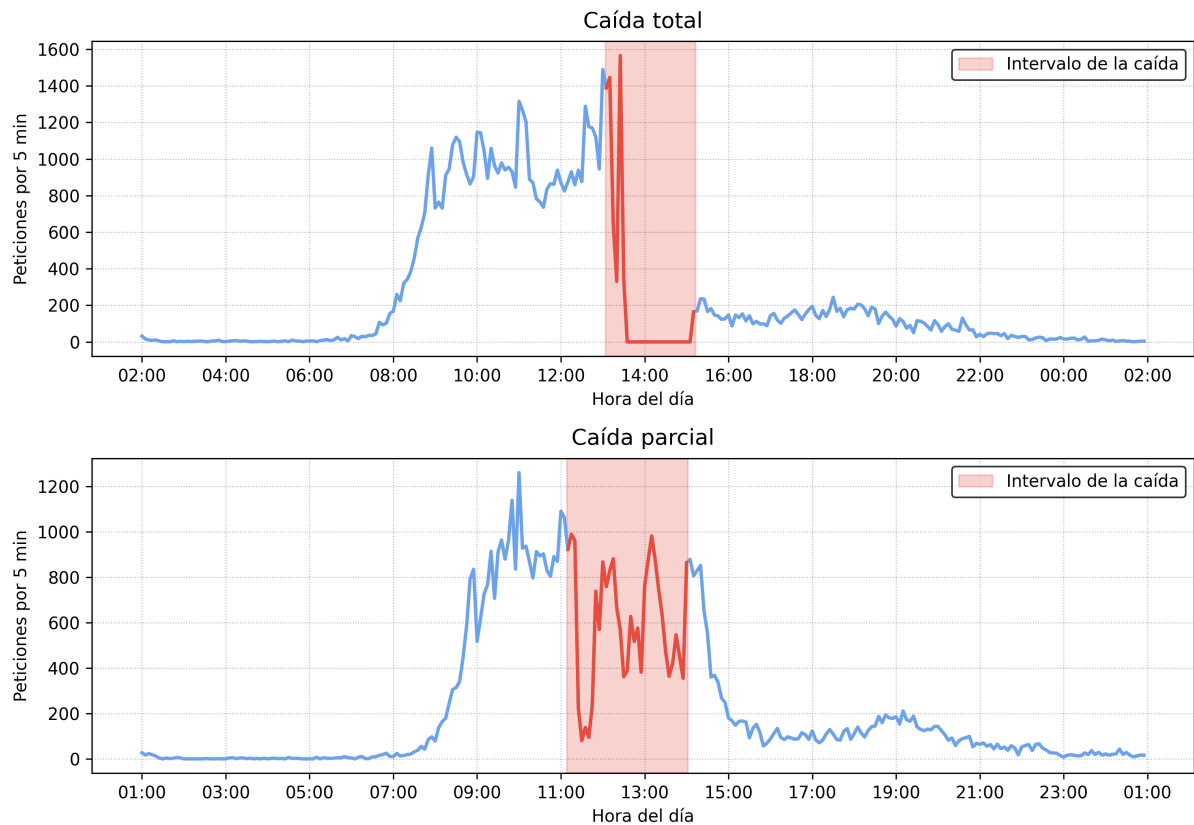


Figure 4.2: Examples of total and partial outages.

The validation and categorization of the anomalies required the implementation of a custom viewer. An interface based on Plotly was built with the following features:

- Rendering of the time series.
- Visualization of the incidents obtained from the chat.
- Creation of anomaly bands.
- Comparison with the traffic estimated by the baseline model.

Figure 4.3 shows the web interface of the incident viewer. The program plots both the actual and the estimated value. It also marks possible outliers with crosses (total outages) or triangles (partial outages). The red and blue bands delimit the outliers marked by the researchers.



Figure 4.3: Incident viewer.

Using this tool, 355 outages were annotated. As can be seen in Figure 4.4, most of these interruptions are total outages.

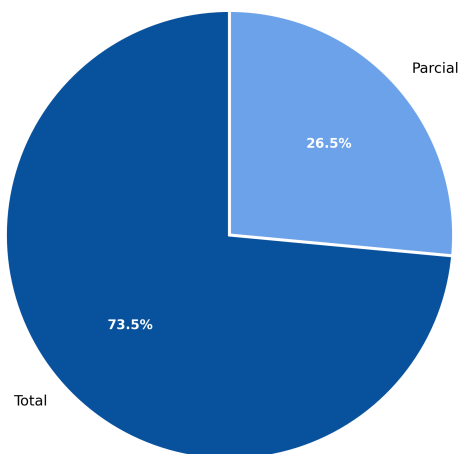


Figure 4.4: Distribution of outage types.

Total outages are easily identifiable, since it is highly unlikely that the system would reach absolute zero during working hours. As will be seen in the next section, partial interruptions give rise to more uncertainty, and the cause of their behavior cannot always be determined.

4.1.3 Construction of exogenous variables

The nature of request traffic is strongly conditioned by external factors that none of the models can infer on their own. For example, Figure 4.5 compares two working Wednesdays with a holiday Wednesday.

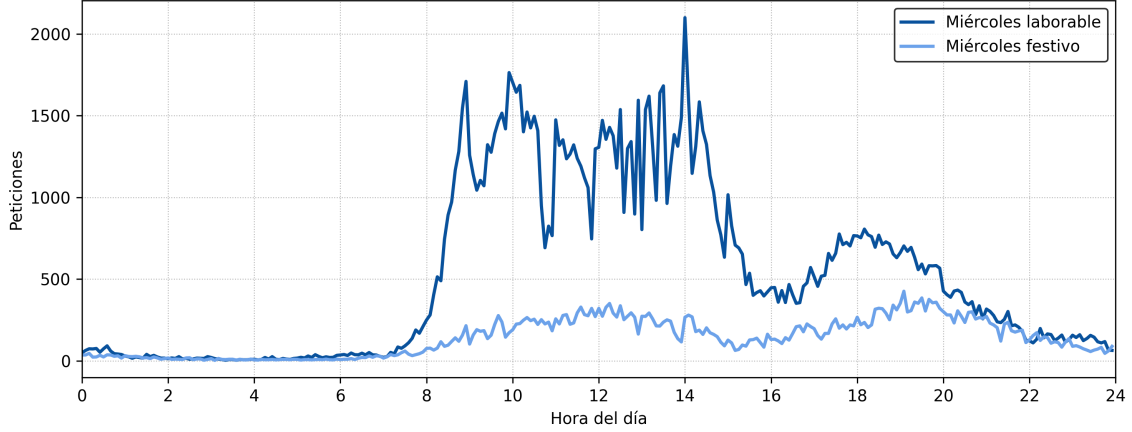


Figure 4.5: Comparison of query request volume between a working Wednesday and a holiday Wednesday.

To address this problem, this thesis opted for models that support exogenous variables. Specifically, three exogenous variables were constructed:

First, the boolean indicator `festivo` was added, which denotes when a day is a national public holiday. The `holidays` library was used to construct it.

Next, the day of the week and the time of day were modeled. Instead of using the usual ranges for hours (0-23) and days of the week (0-6), they were transformed using sines and cosines. The reason for applying these functions is to avoid abrupt jumps: if the hours are not transformed, the model may interpret Sunday (day 6) as being far from Monday (day 0).

$$\text{dow_sin} = \sin\left(\frac{2\pi \cdot d}{7}\right), \quad \text{dow_cos} = \cos\left(\frac{2\pi \cdot d}{7}\right) \quad (4.1)$$

Where d is the day of the week represented as an integer value between 0 (Monday) and 6 (Sunday).

Similarly, for the time of day, the following encoding was used with a period of 1440 minutes:

$$\text{time_sin} = \sin\left(\frac{2\pi \cdot m}{1440}\right), \quad \text{time_cos} = \cos\left(\frac{2\pi \cdot m}{1440}\right) \quad (4.2)$$

Where m is the number of minutes elapsed since midnight.

After applying all the above transformations, the final dataset has the structure shown in Table 4.3.

Table 4.3: Dataset structure after preprocessing.

Attribute	Data type	Description
Date-time	Timestamp	Time index with a 5-minute periodicity.
y	Integer	Total number of query requests aggregated over the 5-minute interval.
festivo	Boolean	Indicator that takes the value 1 if the day is a Spanish national public holiday (or December 24/31) and 0 otherwise.
dow_sin	Decimal	Sine component of the cyclical encoding of the day of the week.
dow_cos	Decimal	Cosine component of the cyclical encoding of the day of the week.
time_sin	Decimal	Sine component of the cyclical encoding of the time of day.
time_cos	Decimal	Cosine component of the cyclical encoding of the time of day.

4.1.4 Dataset split

After preprocessing, the dataset was split into three subsets: training, validation, and test. Since this is a time series, the split must be done chronologically, since taking random samples would break the temporal continuity of the series and confuse the models:

- The first 40% of the series was reserved for training.
- The next 10% was allocated to validation.
- The final 50% of the series was used as the test set on which the performance of all models was evaluated.

Figure 4.6 shows the result of this partition over the complete series, together with the temporal location of the annotated incidents. As can be seen, unintentionally, most of the incidents fall outside the training set. This will prove beneficial for training the models.

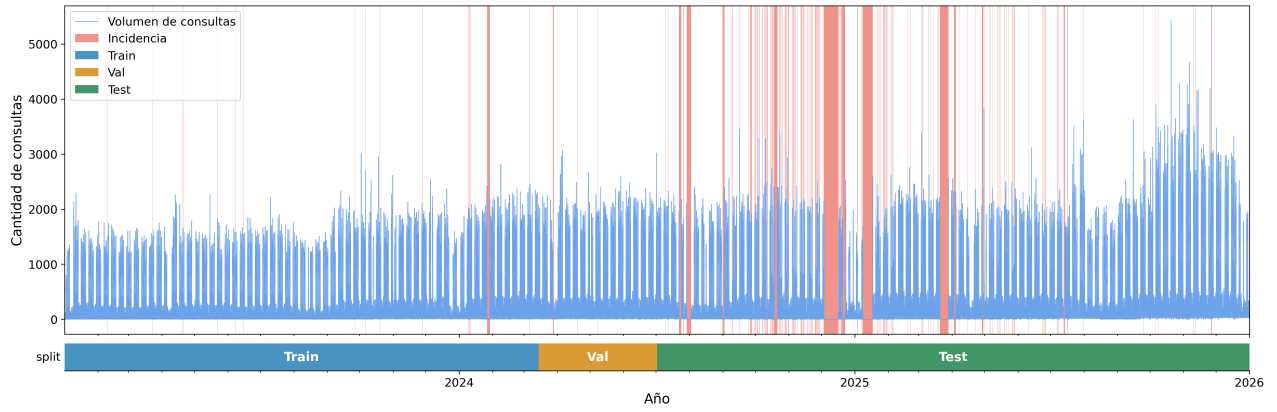


Figure 4.6: Chronological split of the dataset into training, validation, and test sets. The vertical red bands indicate the annotated incidents.

4.2 Exploratory data analysis

Once the dataset had been preprocessed, an exploratory analysis was carried out in order to uncover the nature and patterns of the time series.

4.2.1 Daily pattern

Most working days unfold in a similar way, with an easily recognizable pattern.

Traffic behavior during a typical working day follows an easily recognizable pattern. Figure 4.7 illustrates this over a representative day with no recorded incidents.

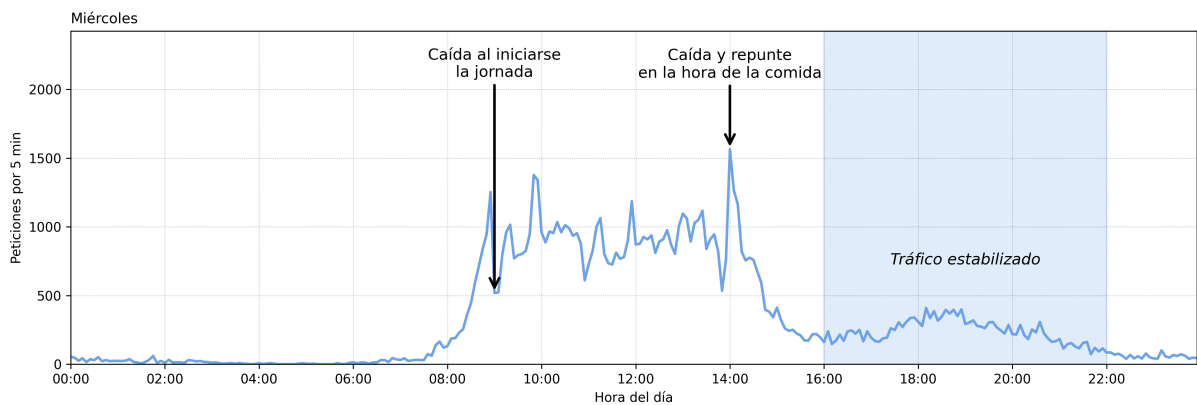


Figure 4.7: Typical daily pattern of query traffic.

Figure 4.7 shows a day from the series in which these patterns can be clearly observed. First, traffic is almost nonexistent until six in the morning, after which it grows until reaching its peak, usually around 9:00. Then, in most cases, there is a brief drop in requests. Traffic then remains level, with

oscillations in query volume. Around 14:00, another abrupt drop usually occurs, followed by an uptick associated with people returning from lunch.

It should be noted that neither of these drops is associated with a system malfunction, but rather with behavior inherent to the data ¹.

After this last peak, traffic declines and then stabilizes, tracing out a small “hill” between 16:00 and 22:00.

4.2.2 System maintenance

A recurring phenomenon in the dataset is scheduled maintenance. These begin at the end of the day on Friday and continue until Monday morning. Figure 4.8 illustrates one of these episodes.

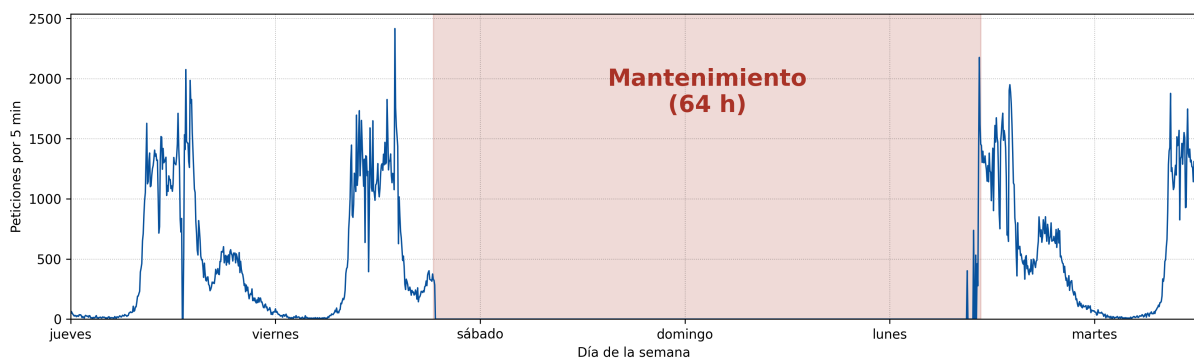


Figure 4.8: Scheduled maintenance from Friday to Monday.

During this type of episode, traffic abruptly drops to zero and remains at that value for approximately 64 hours. Although this is a scheduled event, it is still considered a system anomaly due to its rarity.

4.2.3 Structural change in 2025

In 2025, greater variability in the data and a substantial increase in incidents are observed. Figure 4.9 illustrates this variability on a specific day. In addition, as can be seen in Figure 4.1, at the end of 2025 the volume of requests undergoes an increase, which could be considered a level-shift (LS) outlier.

¹Both patterns were taken into account when labeling the data as anomalous

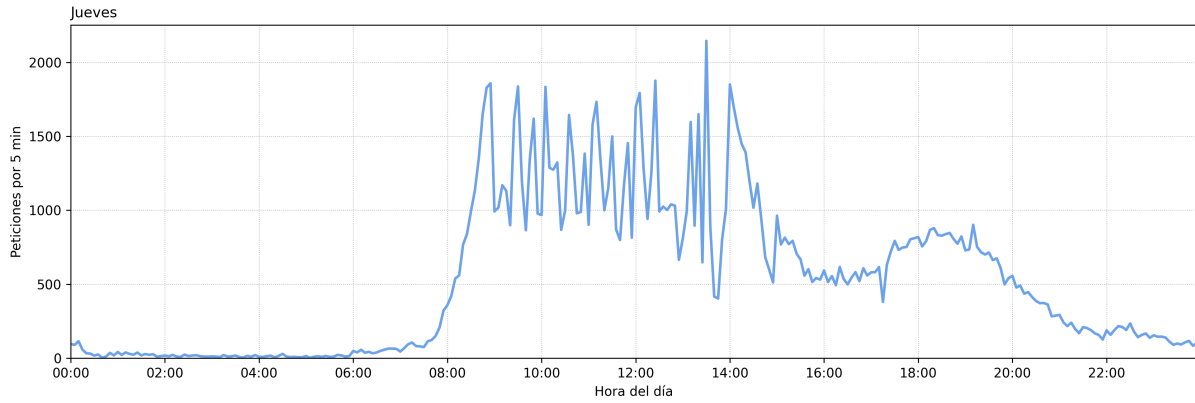


Figure 4.9: Traffic for a representative day after the structural change (with no recorded incidents).

4.2.4 Other aspects analyzed

In addition to the patterns already discussed, reduced traffic was detected on public holidays, along with slight weekly seasonality, whereby each day of the week showed small differences in its behavior.

4.3 Common evaluation framework

To determine the performance of the models, it is first necessary to establish what counts as a hit and what counts as a miss. In this thesis, two different metrics were chosen. First, a custom evaluation was designed, adapted to the use case. Then, in order to cross-check this new metric, the NAB score proposed by Ahmad et al. (2017) was used, which is widely used in the time series anomaly detection literature. This section describes both metrics, as well as the procedure for calculating the detection threshold.

4.3.1 Event-based evaluation

The usual anomaly detection metrics evaluate each instant of the series independently, counting hits and errors discretely. However, this approach does not reflect the typical behavior of alarm systems: the system operator does not mark each instant as anomalous or normal, but rather raises alerts for outages that extend over a period of time. For example, an incident that lasts for an hour and is correctly detected in five of its twelve instants constitutes, from an operational standpoint, a single hit and not twelve.

For this reason, a custom metric was designed, called event-based evaluation, which considers each incident as follows:

- **True positives (TP)**: a real incident is counted as a hit when at least one instant within its time interval has been marked as an anomaly by the model.
- **False negatives (FN)**: real incidents for which no internal instant has been marked as an anomaly.
- **False positives (FP)**: each contiguous sequence of instants marked as an anomaly that does not overlap with any real incident is counted as a single false alarm.

From these counts, precision 4.3, recall 4.4 and the F_1 score 4.5 are obtained.

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (4.3)$$

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (4.4)$$

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \quad (4.5)$$

This way of evaluating does not take into account how long it takes to report outages. Consequently, two complementary quantities are considered: `fp_minutos`, which measures the total false-alarm time to which the operator would be exposed, and `latencia_media`, which measures the average time it takes to raise the alarm for an incident.

4.3.2 Numenta's NAB metric

In order to compare the developed models with those in the literature, it was decided to evaluate the models with the NAB score, proposed by Ahmad et al. (2017). This score is very well established in time series anomaly detection research, and was also devised for real-time data like that used in this thesis.

Numenta's public code was used (Ahmad et al., 2019), which treats anomalies as discrete points. Since the incidents in this thesis were marked using intervals, the first point of each anomalous interval was taken.

The procedure for calculating the NAB score is detailed below:

- 1) First, around each real anomaly a time window is defined whose length is equivalent to 10% of the total length of the series, divided by the total number of anomalies in the data (Ahmad et al., 2017, sec. 3.2). If two windows overlap, they are merged into one.
- 2) Second, each observation is scored taking into account its position relative to the window, such that the earlier it is detected, the higher the score it receives. However, only the first detection counts; the rest are redundant. False positives are subtracted from the score, subtracting less if they are close to the window. If a window is not detected, this also subtracts from the score.

The score can be adjusted to the use case by means of the application profile (Ahmad et al., 2017, sec. 3.2). This application profile consists of parameters that assign a specific importance to each type of error. In this thesis, the standard profile is used, which is the one commonly used in the literature.

- 3) Finally, the score is normalized to the interval $[0, 100]$ using as reference a perfect detector, which would trigger the first alarm exactly at the start of each incident without generating false positives, and a null detector, which would not detect any anomaly (Ahmad et al., 2017, sec. 4.2). In this way, the result is comparable across datasets of different length and number of incidents.

4.3.3 Calculating the detection threshold

All prediction-based models (SARIMAX, DeepAnT and LSTM) use the same detection system. To determine whether an observation is atypical, the value expected by the model is predicted and its anomaly score (2.22) is calculated.

An observation is labeled as anomalous when its anomaly score exceeds a certain threshold. This threshold affects the model's precision and recall: a low threshold causes more false positives, while a high value leaves most real incidents undetected.

In order to find a suitable threshold, an optimization procedure was designed based on the validation set². Specifically, Brent's method is used, implemented in `scipy.optimize.minimize_scalar`. The procedure explores the interval $[\min(s_t), \max(s_t)]$ and returns the value that maximizes the evaluation metric over the validation scores.

Since two different metrics are used (event-based and NAB), an optimized threshold is generated for each one.

Finally, a restriction is added to anomaly detection: only observations whose predictions are greater than the observed value are marked. In this way, the models only detect system outages. This decision was made in light of the specific objective of this thesis, which is to detect interruptions in request traffic.

4.4 Models

4.4.1 Baseline model

Before training the complex models, a reference detector was developed to establish a baseline for comparison. Its construction is straightforward and is based on the actual detector used in the company's application.

²Optimizing the threshold using observations from the test set could be seen as dishonest.

The prediction is obtained by calculating the average of the observed value at the same hour over the last 5 non-holiday weekdays. For example, to label an observation from a Monday at 19:00, the average value of the last 5 Mondays at 19:00 is computed. Once this average value is obtained, the following detection rules are applied.

- Only instants belonging to working days (Monday through Friday), within the time window between 7:00 and 22:00, and that do not coincide with national public holidays, are evaluated.
- If the forecast is below a minimum threshold (100 requests in this thesis), the interval is considered to lack relevant activity and is discarded.
- Any observation in which traffic is exactly zero is marked as a total outage.
- Any observation in which the observed traffic is below a threshold obtained as previously described is marked as a partial outage.

4.4.2 SARIMAX model

The second model developed belongs to the classical ARIMA family of models, described in the theoretical framework. Specifically, it is a SARIMAX, which extends ARIMA with a seasonal component and exogenous variables. The motivation for including it in this thesis is to compare a classical approach with modern deep learning techniques.

Data preparation

Unlike the rest of the models, SARIMAX was not fit on the series with 5-minute periodicity. Instead, the data was grouped into 30-minute periods. This is due to a technical limitation, since lowering the periodicity causes RAM and CPU usage to spike during model fitting.

As for the exogenous variables, not all five built variables were incorporated. The variables `time_sin` and `time_cos` were discarded because the information they provide is already captured by the model's seasonal parameters.

Stationarity and seasonality analysis

In order to justify the use of SARIMA³, an analysis of the seasonality of the data was performed. To do so, the series was decomposed into trend, seasonality and residual, as shown in Figure 4.10.

³If the series did not exhibit seasonality, the seasonal components of SARIMA would be redundant.

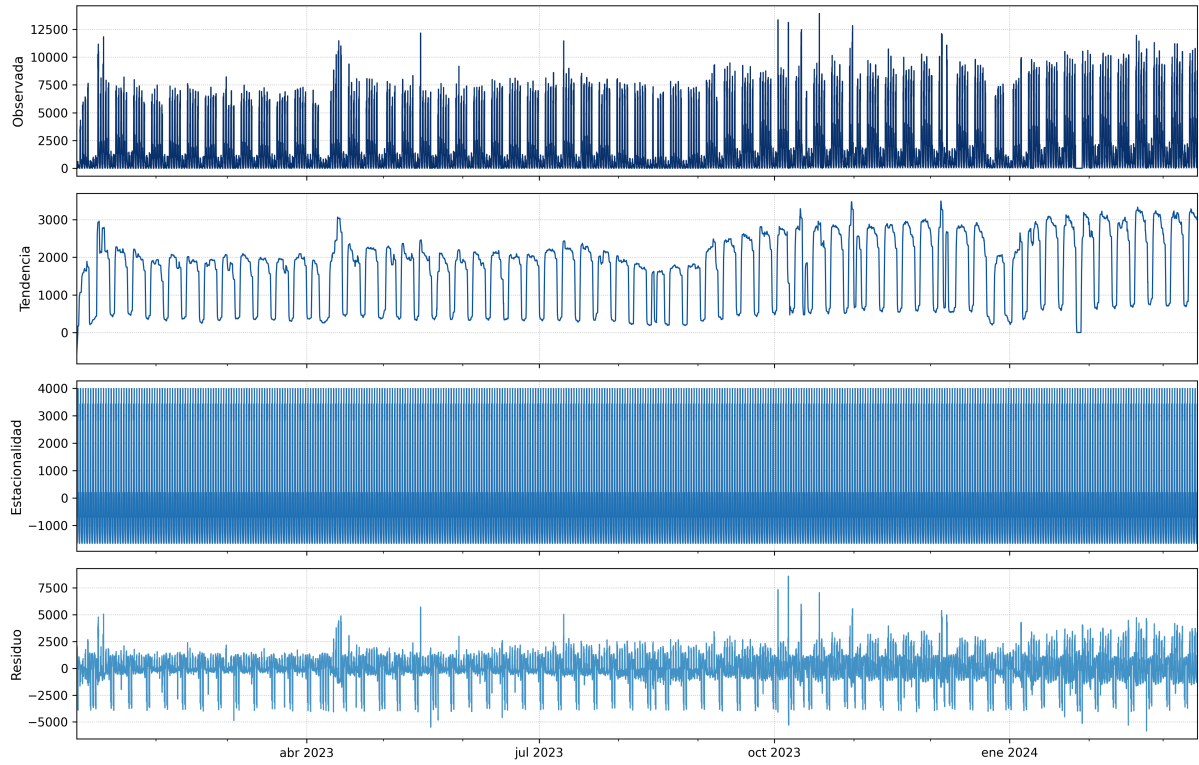


Figure 4.10: Additive seasonal decomposition of the series with period $m = 48$.

When calculating the variance attributable to each component, it was found that approximately 60% of the observed variance was explained by the seasonal component, which allows us to speak of strong seasonality. In addition, the ACF and PACF correlograms of the series up to 72 lags were inspected, shown in Figure 4.11.

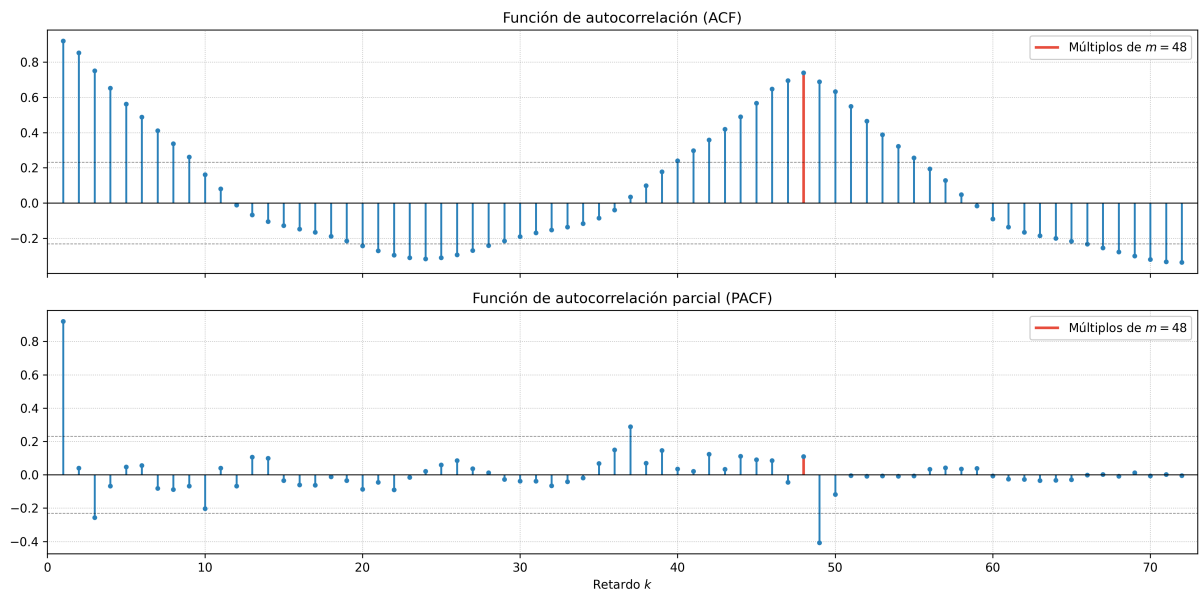


Figure 4.11: ACF and PACF correlograms of the series up to 72 lags.

The presence of pronounced peaks at multiples of the daily period confirms the daily seasonality of the series. It is worth noting that peaks were also observed at multiples of $7 \cdot 48 = 336$, which indicate an additional weekly seasonality. However, the classical model cannot model both seasonalities simultaneously⁴.

As for stationarity, changes in trend and variance had already been observed in the exploratory analysis. Therefore, the series was differenced as described in the theoretical framework (2.8). After this transformation, the result was validated using hypothesis tests. Specifically, the ADF test and the KPSS test were applied, both to check whether a series is stationary or not. Both tests indicated that the series was stationary after the transformation.

Model fitting

The selection of the orders $(p, d, q) \times (P, D, Q)_m$ was performed automatically using the `AutoARIMA` algorithm from the `statsforecast` library. This algorithm is a Python adaptation of the R function `auto.arima()`, which implements the Hyndman-Khandakar procedure (Hyndman & Athanasopoulos, 2018, sec. 8.7).

The fitting was performed on the concatenated training and validation sets, in order to cover as much historical data as possible. Of all the models, SARIMAX turned out to be the longest to train, since the algorithm runs on a single CPU processor.

Prediction quality

After fitting the model, the quality of its forecasts on the test set was checked using different metrics. Additionally, the model was evaluated using the test set without anomalies to give a more faithful measure of its predictive performance.

Metric	Full test	Test without incidents
MAE	1889.43	1684.67
RMSE	2467.50	2165.06
MAPE	4.977	5.025
sMAPE	1.099	0.993
Bias	-1342.86	-1089.39
R^2	0.237	0.429

Table 4.4: SARIMAX prediction metrics on the test set.

⁴Instead, more modern decomposition models such as Meta’s Prophet can be used.

Table 4.4 shows that the model barely reaches a coefficient of determination of 0.43 when excluding anomalies. The markedly negative bias indicates that the model tends to systematically overestimate the actual value.

Additionally, Figure 4.12 shows the actual and predicted values over a representative week of the test set. It can be observed that the model has not captured the day-of-week exogenous variable well, since it produces the same traffic every day.

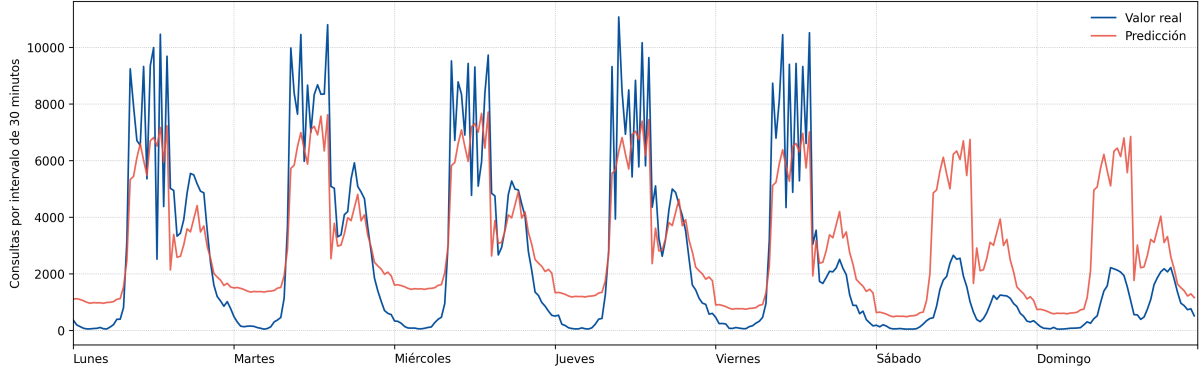


Figure 4.12: Actual values and SARIMAX predictions.

Ultimately, classical models such as SARIMAX fall short when faced with series exhibiting the variability characteristic of web traffic. On one hand, it has been shown to have difficulty capturing multiple seasonalities simultaneously. On the other hand, its linear nature prevents it from reproducing the abrupt changes and high variance that characterize this type of data.

4.4.3 DeepAnT model

The second model implemented is an adaptation of the DeepAnT architecture (Munir et al., 2019), explained earlier in the theoretical framework.

Architecture

The network uses the architecture proposed by the original authors. Specifically, the network is composed of the following elements:

- Input layer that receives a window of W instants and n features.
- First convolutional block: `Conv1D` layer with 32 filters and a kernel of size 3, `ReLU` activation function, followed by a `MaxPooling1D` layer of size 2.
- Second convolutional block, identical to the previous one.

- Flatten layer that flattens the feature map.
- Dense layer with a single neuron that predicts the value immediately following the window.

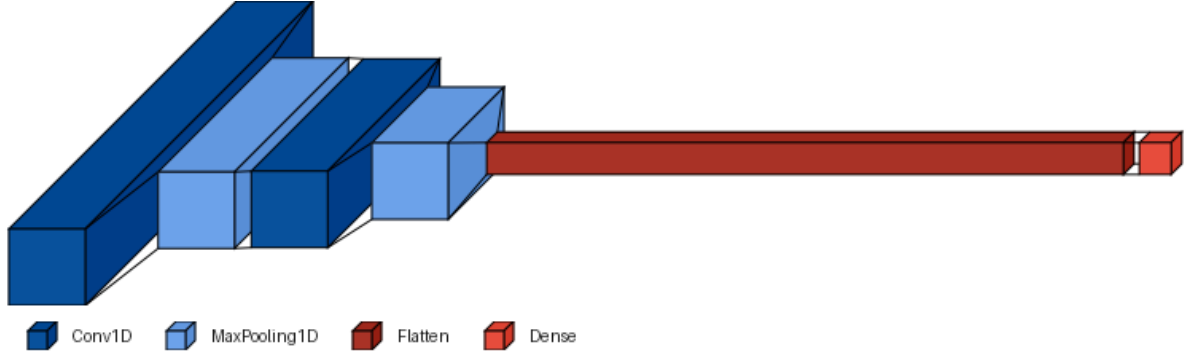


Figure 4.13: Architecture of the implemented DeepAnT model.

Mean absolute error (MAE) was used as the loss function and stochastic gradient descent (SGD) as the optimizer, again following the original proposal of the paper.

Data preparation

A very common practice in training neural networks is to normalize the data (Aggarwal, 2023, sec. 2.7.2). This technique transforms the input variables so that they share a similar scale. This prevents some variables from carrying more weight than others simply because they have a larger scale. Additionally, it helps some optimization algorithms converge, such as gradient descent.

The observations were normalized using `MinMaxScaler` scaling, following formula 4.6 (Aggarwal, 2023, p. 64).

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (4.6)$$

Equation 4.6 was applied exclusively to the training set to prevent the rest of the dataset from biasing the scaling.

Subsequently, sliding windows of size $W = 288$ were built, equivalent to 24 hours of history at 5-minute resolution. Each window acts as input to the network, and its corresponding label is the value of the immediately following instant.

Unlike the SARIMAX model, DeepAnT incorporated all the available exogenous variables: `fest_ivo`, `dow_sin`, `dow_cos`, `time_sin` and `time_cos`. These variables are especially important for the convolutional network, since, unlike the LSTM network, the convolutional network has no historical context and bases its predictions solely on the data within the window W .

Model training

The model was trained for 200 epochs, with a historical window of 288 steps spanning a full day. To speed up training, mixed precision (NVIDIA Corporation, 2023) was used, a technique that reduces the precision of tensors in the network's intermediate computations in order to take advantage of the GPU's tensor cores⁵.

In addition, a `batch_size` of 256 was chosen. Although the physical machine's memory allowed for a larger batch size, it was observed that this had a detrimental effect on the predictions.

The evolution of the loss during training is shown in Figure 4.14. The training curve decreases monotonically, while the validation curve decreases while oscillating. It can be seen that although the loss continues to decrease, the validation error stabilizes around epoch 50. This suggests that, from epoch 50 onward, the model will not improve its forecasts.

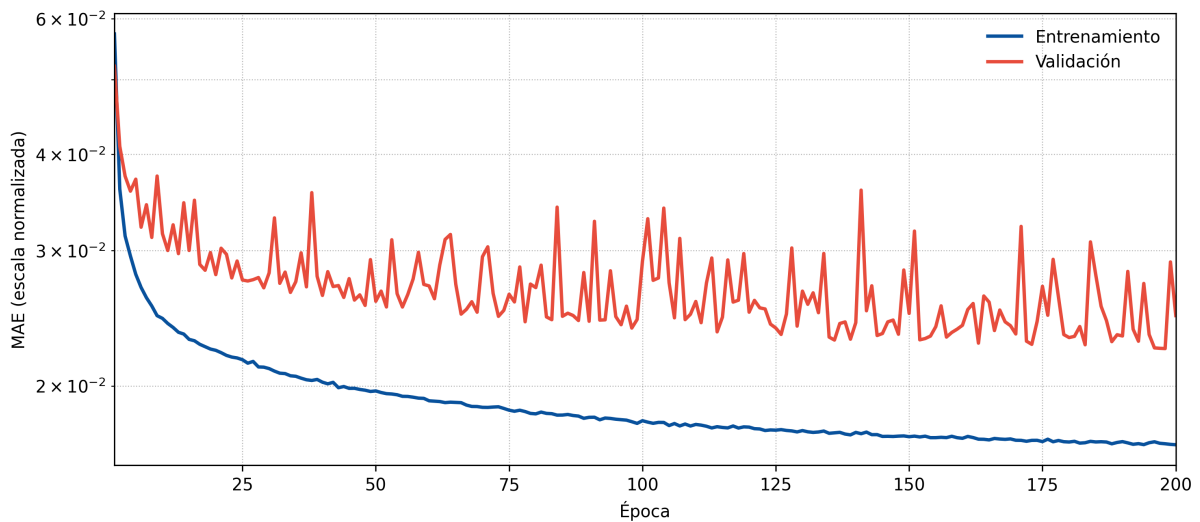


Figure 4.14: Evolution of the training and validation errors during DeepAnT training.

Prediction quality

The model's prediction quality was evaluated on the training set with and without anomalies. This second examination provides a more faithful measure of the model's purely predictive performance.

⁵Tensor core units have been available since NVIDIA's Volta architecture.

Metric	Full test	Test without incidents
MAE	96.53	82.45
RMSE	202.47	169.86
MAPE	0.785	0.758
sMAPE	0.842	0.680
Bias	5.95	21.52
R^2	0.828	0.881

Table 4.5: DeepAnT prediction metrics on the test set.

Table 4.5 shows that the model has a coefficient of determination of 0.88 when excluding anomalies. This metric drops when we introduce system outages, as would be expected. The slightly positive bias indicates that the model tends to underestimate the actual value⁶.

Additionally, Figure 4.15 shows the actual and predicted values over a representative week of the test set.

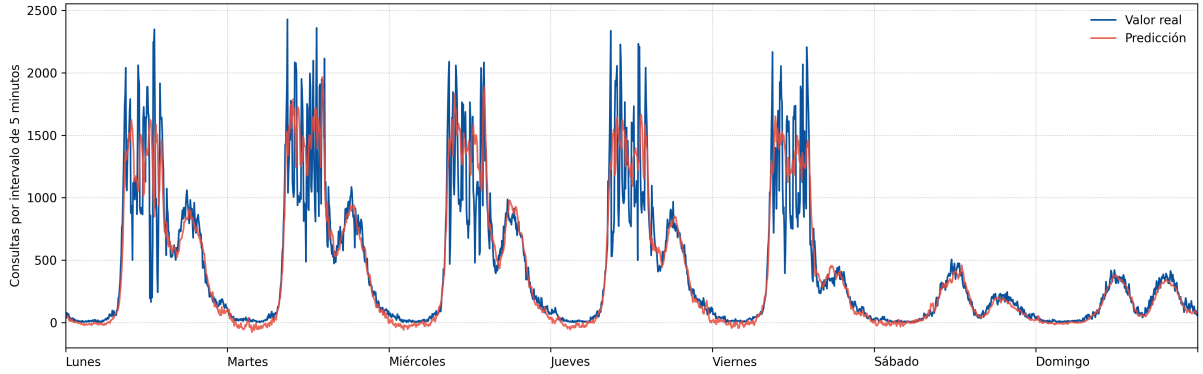


Figure 4.15: Actual values and DeepAnT predictions.

It can be seen that the network captures the daily and weekly patterns of request traffic. It can also be seen that the network does indeed tend to underestimate the actual observations.

4.4.4 LSTM model

The third and final predictive model implemented is a long short-term memory recurrent network, inspired by the procedure proposed by Malhotra et al. (2015).

⁶The reason for this behavior is that the model was trained on contaminated data. Outages pull the predictions downward.

Data preparation

As with the previous model, the data was prepared by applying `MinMaxScaler` normalization, and sliding windows of the same size $W = 288$ were built, also keeping the same exogenous variables. The difference with respect to DeepAnT is that the procedure of Malhotra et al. (2015) requires training the network with data free of anomalies. Consequently, all windows contaminated with outliers were discarded ⁷.

Architecture

The architecture used consists of the following layers:

- Input layer that receives a window of $W = 288$ instants and n features.
- First LSTM layer with 64 units and full sequence return (`return_sequences=True`).
- Dropout layer with probability 0.2 to mitigate overfitting.
- BatchNormalization layer that stabilizes the distribution of the activations.
- Second LSTM layer with 32 units, without sequence return.
- Dropout + BatchNormalization block, analogous to the previous one.
- Dense layer with $L = 10$ neurons, responsible for predicting the next 10 steps.

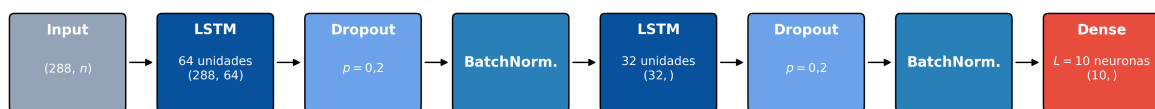


Figure 4.16: Architecture of the implemented LSTM model.

MAE was kept as the loss function; however, the optimizer was replaced with Adam, due to its better performance during initial testing.

⁷If only the incident intervals were discarded, abrupt jumps would occur in the data. Therefore, the entire window must be discarded even if more data is lost.

Multi-step prediction

Unlike DeepAnT, which predicts only the next instant, the LSTM network is trained with a multi-step horizon of $L = 10$ values. This horizon forces the model to capture more extensive temporal dependencies. However, only the one-step prediction is used to compute the anomaly score, thus maintaining compatibility with the common evaluation framework.

This decision deviates slightly from the original procedure proposed by Malhotra et al. (2015). The paper uses the Mahalanobis distance over the multidimensional error vector. However, during development it was observed that this approach led to worse results on this dataset, and it was therefore ultimately discarded.

Model training

Initially, the model was trained in a manner similar to DeepAnT, using 200 epochs. However, when examining the training curves (4.17), it was observed that the validation error increased around epoch 66.

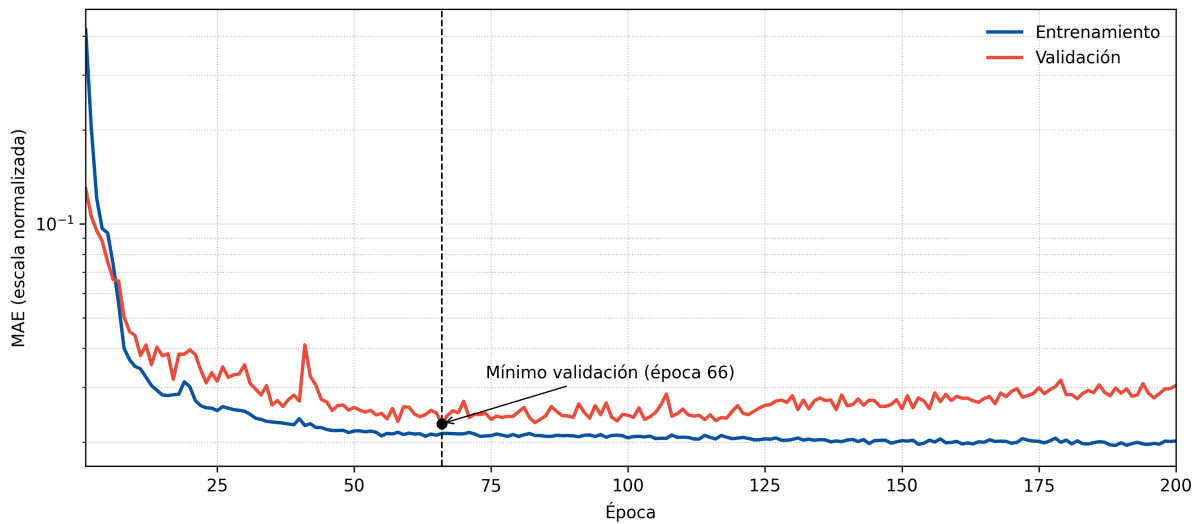


Figure 4.17: Training curves of a preliminary LSTM run with 200 fixed epochs.

To mitigate this effect, the early stopping technique was used, which stops training if no improvement is observed. Specifically, the validation loss was monitored with a patience of 10 epochs, and once the stopping was triggered, the weights corresponding to the best observed epoch were restored.

After incorporating early stopping, training was relaunched. The resulting curves are shown in Figure 4.18.

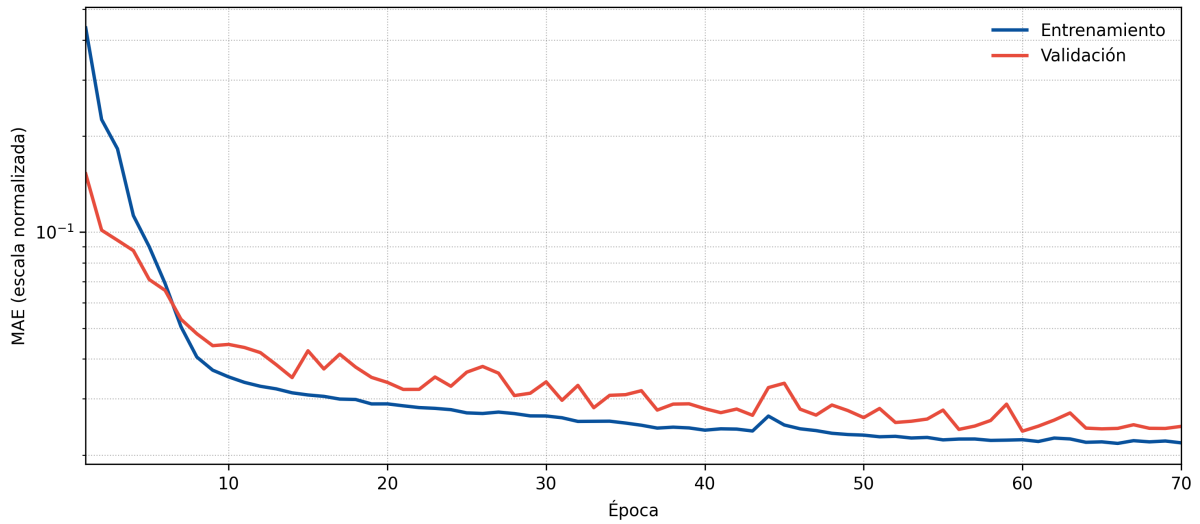


Figure 4.18: Training curves of the LSTM with early stopping enabled.

Prediction quality

As with DeepAnT, the prediction metrics of the network were calculated on the test set. The results are shown in Table 4.6.

Metric	Full test	Test without incidents
MAE	77.72	72.40
RMSE	176.26	160.39
MAPE	0.426	0.392
sMAPE	0.554	0.342
Bias	-12.08	-4.48
R^2	0.870	0.894

Table 4.6: LSTM prediction metrics on the test set.

A slight improvement can be observed in all evaluation metrics. The negative bias indicates that the network now tends to overestimate; however, it has less bias than the previous model.

Figure 4.19 shows the predictions of the LSTM network over a week of the test set.

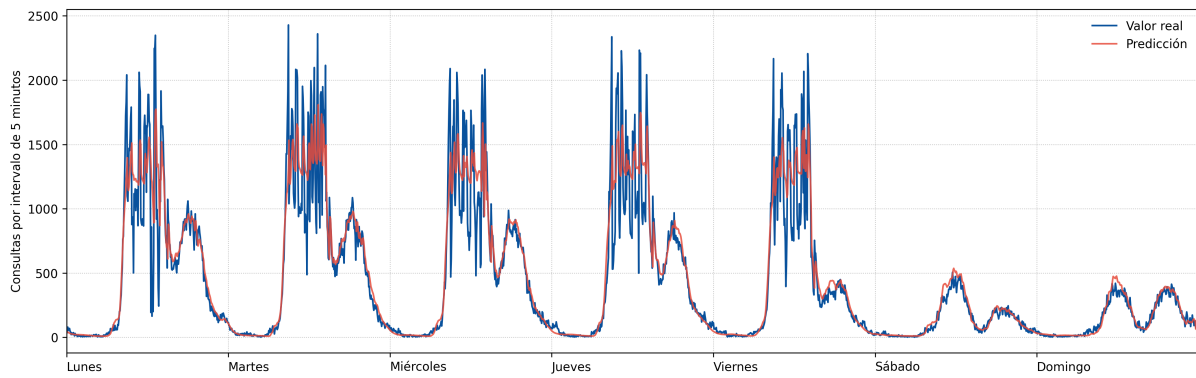


Figure 4.19: Actual values and LSTM predictions over a week of the test set.

As with DeepAnT, it can be observed that the network captures the daily and weekly patterns well.

4.5 Hyperparameter tuning

The choice of hyperparameters was based on several training experiments. These experiments were carried out with the help of the Optuna library, which automates the search and adjusts the hyperparameters so as to maximize the detection F_1 score. In addition, it stores the results in a SQLite database in order to keep a historical record of the trained models.

Figure 4.20 shows the importance that Optuna assigns to each hyperparameter. In the case of DeepAnT, it determined that window size was the factor that most influenced the results.

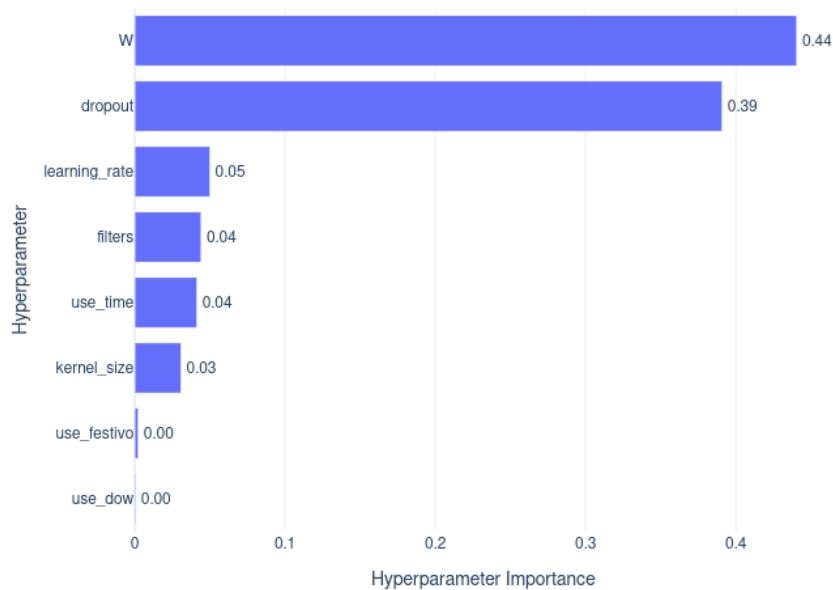


Figure 4.20: Optuna chart showing the importance of each hyperparameter.

The procedure was applied exclusively to the DeepAnT and LSTM models, since SARIMAX already has an automated search mechanism provided by `AutoARIMA`.

4.6 Model evaluation

Once the models were fitted, their comparative evaluation on the test set was carried out. For this purpose, the two metrics described in the common evaluation framework were used: the custom event-based metric and the NAB score. The results are collected in Table 4.7⁸.

Model	TP	FN	FP	Recall	F_1	NAB
Baseline model	116	142	64	0.450	0.530	15.03
SARIMAX	103	155	397	0.399	0.272	-78.01
DeepAnT	154	103	70	0.599	0.640	61.11
LSTM	166	91	69	0.646	0.675	65.53

Table 4.7: Detection metrics of the models on the test set.

The LSTM stands out as the best-performing model on both metrics, closely followed by DeepAnT. The baseline model, despite its simplicity, clearly outperforms SARIMAX, which closes out the ranking with the worst F_1 and a negative NAB score. This last point is especially striking, since it indicates that SARIMAX's behavior is worse than that of a detector that never raises alarms.

To show the behavior of each model, Figure 4.21 depicts the alerts issued over a day with several incidents from the test set. Each model appears with a different marker.

⁸The TP, FN, FP, recall and F_1 columns are computed according to the custom event-based metric described in the common evaluation framework.

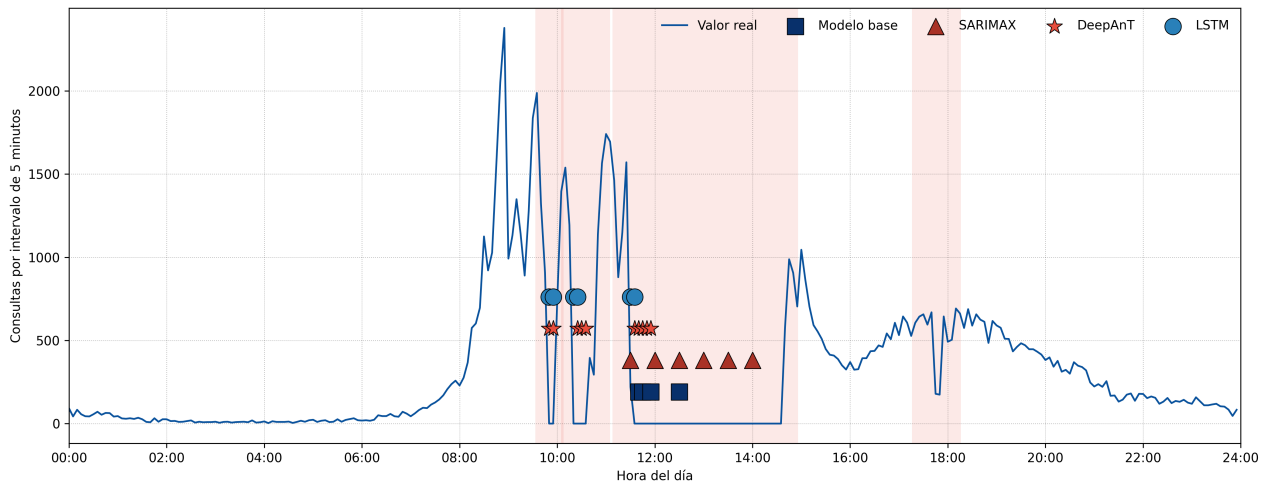


Figure 4.21: Detections of the four models over a day with multiple incidents from the test set.

First, it can be observed that the only models that detected the first incident were the ones based on neural networks. In addition, DeepAnT and SARIMAX are seen to be the models that issue the most detections during the anomalous intervals. The LSTM raises few alarms, but all of them within the real intervals. No model is able to detect the last incident of the afternoon, which shows the limitations of the anomaly score and of prediction-based models. Since the last interval already has low traffic to begin with, an outage does not stand out as much as it would during business hours.

To complete the comparison from the operator's point of view, Table 4.8 shows two additional quantities: the median detection latency⁹, which measures how long it takes each model to raise the first alarm for a real incident, and the total false-alarm time, which quantifies the accumulated minutes of alerts outside of any anomalous interval.

Model	Median latency	Total false-alarm time
Baseline model	32 min	13 h 10 min
SARIMAX	26 min	442 h 30 min
DeepAnT	21 min	11 h 25 min
LSTM	22 min	6 h 55 min

Table 4.8: Median detection latency and total false-alarm time of the models on the test set.

DeepAnT and LSTM are the most responsive models, with median latencies of around 20 minutes. SARIMAX and the baseline model are somewhat above that, without seriously compromising the

⁹The median is used instead of the mean since it is more resilient to outliers. For example, if there were a 3-day maintenance period and the outage were detected upon the recovery of normal traffic, the mean detection time would spike.

reaction time to a real incident.

Regarding the total false-alarm time, the LSTM network is the most efficient option, with less than seven hours accumulated over the year and a half of the test set. SARIMAX, by contrast, accumulates more than 442 hours of false alarms, equivalent to about 18 days of alerts. This figure is untenable for an operator and is reason enough to exclude SARIMAX from the final detection system.

4.6.1 Comparison with the literature

As a rough point of reference, the authors of NAB publish a comparative table with the results of several detectors on their official benchmark (Ahmad et al., 2019). Table 4.9 collects those scores.

Detector	NAB <i>Standard Profile</i>
Perfect detector	100.0
ARTime	74.9
Numenta HTM	70.5
CAD OSE	69.9
earthgecko Skyline	58.2
KNN CAD	58.0
Relative Entropy	54.6
Random Cut Forest	51.7
Twitter ADVec v1.0.0	47.1
Windowed Gaussian	39.6
Etsy Skyline	35.7
Bayesian Changepoint	17.7
EXPoSE	16.4
Random detector	11.0
Null detector	0.0

Table 4.9: NAB score of various reference detectors on Numenta’s official benchmark (Ahmad et al., 2019).

As a rough guide, the results obtained in this thesis (DeepAnT: 61.11, LSTM: 65.53) would fall in the upper half of the table, above detectors such as earthgecko Skyline or KNN CAD and below CAD OSE, Numenta HTM or ARTime.

However, this comparison should be taken with a grain of salt. The models in the table were evaluated

on the NAB corpus, a set made up of 58 heterogeneous time series (AWS metrics, Twitter traffic, sensor data, etc.) with characteristics very different from the use case of this thesis. Aspects such as the period of the series, the nature of the anomalies, or the proportion of clean versus anomalous data have a decisive influence on the score. For this reason, these figures should be used only as a general reference and not as a strict comparison.

4.7 Real-time detection application

As a conclusion to the development of this thesis, a real-time monitoring application was built that integrates the trained detectors. The tool replaces the rudimentary alarm system originally used by the company. The SARIMAX model was omitted due to its poor detection results.

4.7.1 Usefulness and role in operations

The dashboard was designed to reduce false positives compared to the previous system. The operational problem was not the absence of alarms, but their excess: when a tool frequently triggers alerts, the operator ends up ignoring them, and real outages go unnoticed.

The application addresses this problem by combining the three detectors through a consensus criterion and centralizing on a single screen the instantaneous value of the metric, the expected estimate, the status of each detector, and a chronological history of confirmed incidents.

4.7.2 Architecture and operation

The tool was implemented as a Python web application with the `Dash` framework. In this way, part of the visualization code from the incident viewer is reused, and the interface is exposed through a conventional browser.

The system processes each new observation in four stages. First, an ingestion module supplies the data at a controlled rate; the prototype replays the test set, whereas in production the source would be the company's real telemetry. Second, the three models (baseline model, DeepAnT, and LSTM) receive the observation within a sliding window and return their prediction and individual verdict. Next, a consensus module combines the verdicts: a band is only promoted to a confirmed incident when at least two models agree in flagging it. Finally, the dashboard updates the information visible to the operator and logs the incident in the history.

4.7.3 Interface description

Figure 4.22 shows a screenshot of the dashboard during a simulation. The interface is organized into four blocks.

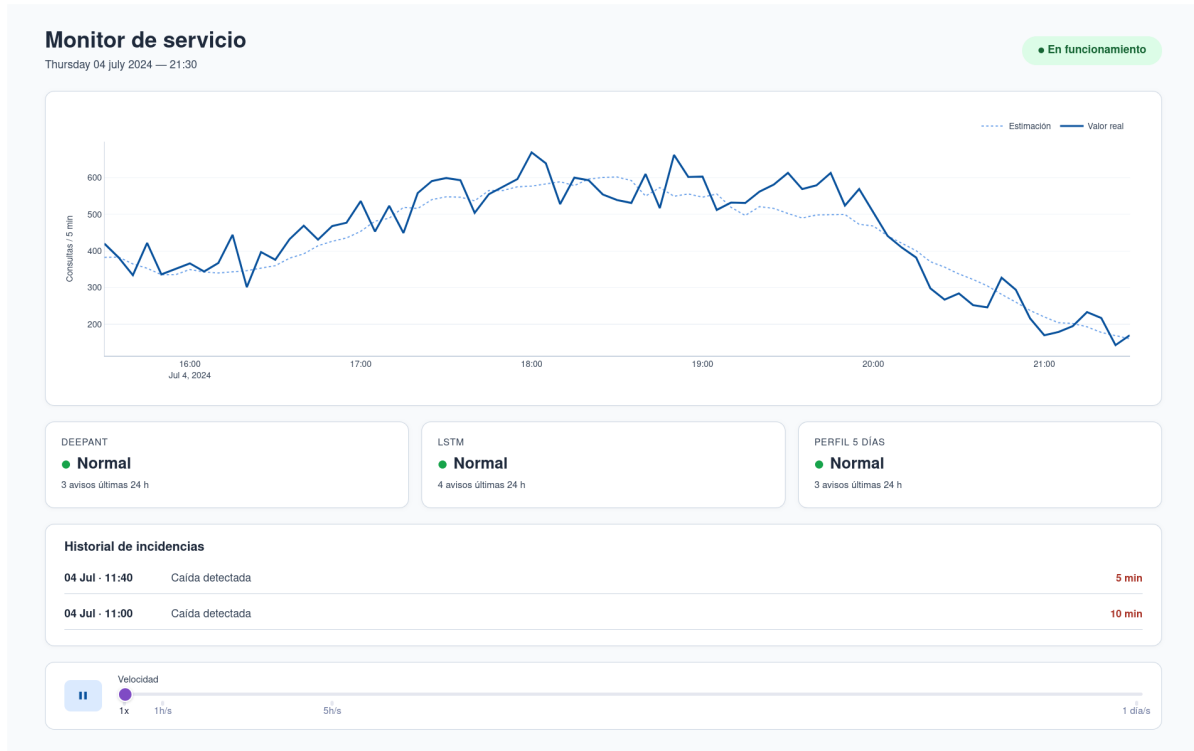


Figure 4.22: Dashboard of the real-time detection application.

In the top block are the monitor title, the timestamp of the simulated instant, and a status label that switches from green (“Running”) to a prominent warning when an incident is confirmed.

The main chart represents the data flow over a moving window of the last few hours. The solid line corresponds to the actual value, and the dashed line to the average estimate of the three models.

Below the chart are three individual panels, one per model (DeepAnT, LSTM, and the baseline model). Each panel shows the detector’s verdict (*Normal* or *Alert*) and the number of alerts issued in the last twenty-four hours. Showing the detectors separately makes diagnosis easier when a particular model reports more alerts than usual.

The “Incident history” block collects the most recent confirmed outages in a table. For each incident, the detection timestamp, a brief description, and the latency relative to the estimated start of the band are shown.

Finally, at the bottom there is a simulation speed control. It allows the test set to be replayed at different speeds and execution to be paused in order to inspect specific episodes. This control would not be part of the production deployment.

Chapter 5

Conclusions

5.1 Conclusions

This thesis has developed a complete anomaly detection system for web traffic time series, based on a real case from a company with a rudimentary alarm system. All the planned phases have been addressed, from the preprocessing of the dataset to the integration of the models into a real-time monitoring application.

Regarding the model comparison, the LSTM network stood out as the best detector in both the event-based metric and the NAB score. DeepAnT achieves very similar results and, moreover, requires a noticeably shorter training time. Without being a strict comparison, both models would fall in the upper half of Numenta's official benchmark, which suggests that their performance is reasonable within the state of the art. The baseline model, despite its simplicity, has proven to be a solid reference, useful for ruling out implementations that fail to outperform a simple historical average. SARIMAX, on the other hand, turned out to be clearly insufficient: its inability to model two simultaneous seasonalities, combined with the linear nature of the ARIMA family, leaves it well behind the rest and warrants excluding it from the final system.

Finally, the real-time detection application integrates the three selected models (baseline model, DeepAnT and LSTM) and combines them by means of a consensus criterion, requiring that at least two models trigger simultaneously in order to consider an interval an incident. This strategy takes advantage of the complementarity between the statistical approach and the deep learning-based approaches, and reduces the number of individual false positives from each detector.

5.2 Future work

Throughout the development process, several lines of improvement have been identified that could continue this thesis in the future.

Reducing latency

One direct avenue for improvement would be to reduce the periodicity of the series from 5 minutes to 1 minute, in order to shorten the detection latency. This approach was attempted during development, but was discarded for two reasons. On one hand, the training cost rose sharply, making hyperparameter search unfeasible. On the other hand, the models trained with this periodicity produced lower-quality predictions, which in practice worsened latency by delaying the triggering of alarms. Exploring architectures that solve this problem remains an open avenue.

More cutting-edge models

The architectures used in this thesis (LSTM and CNN) are representative of the state of the art prior to the emergence of *Transformers*, which constitute the dominant paradigm today. Hybrid models such as those discussed in the related work section could also be explored.

Better labeling

The quality of the labeling carried out in this thesis is limited by the degree of familiarity with the system. One possible improvement would be to base the labeling on the users' incident ticketing system. Another option would be to label the data together with an experienced team familiar with the system.

References

- Aggarwal, C. C. (2017). *Outlier analysis*. Springer. <https://doi.org/10.1007/978-3-319-47578-3>
- Aggarwal, C. C. (2023). *Neural networks and deep learning: A textbook* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-031-29642-0>
- Ahmad, S., Lavin, A., Purdy, S., & Agha, Z. (2017). Unsupervised real-time anomaly detection for streaming data [Online Real-Time Learning Strategies for Data Streams]. *Neurocomputing*, 262, 134–147. <https://doi.org/https://doi.org/10.1016/j.neucom.2017.04.070>
- Ahmad, S., Lavin, A., Purdy, S., Agha, Z., Danforth, I., Lewis, M., Scheinkman, L., Taylor, M., Gokhale, J., Silver, T., Smirnov, M., Marshall, A., Borgne, M. L., Takashina, Y., Kibish, S., ish-vlad, Baranski, C., Crowder, R., m.fab, ... breznak. (2019, December). *Numenta/nab: V1.1* (Version v1.1). Zenodo. <https://doi.org/10.5281/zenodo.3571294>
- Box, G. E. P., Jenkins, G. M., & Reinsel, G. C. (2008). *Time series analysis: Forecasting and control* (4th ed.). John Wiley & Sons.
- Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). Lof: Identifying density-based local outliers. *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 93–104. <https://doi.org/10.1145/342009.335388>
- Brockwell, P. J., & Davis, R. A. (2016). *Introduction to time series and forecasting* (3rd ed.). Springer. <https://doi.org/10.1007/978-3-319-29854-2>
- Chang, I., Tiao, G. C., & Chen, C. (1988). Estimation of time series parameters in the presence of outliers. *Technometrics*, 30(2), 193–204. <https://doi.org/10.1080/00401706.1988.10488367>
- Chen, C., & Liu, L.-M. (1993). Joint estimation of model parameters and outlier effects in time series. *Journal of the American Statistical Association*, 88(421), 284–297. Retrieved June 14, 2014, from <http://www.jstor.org/stable/2290724>
- Fox, A. J. (1972). Outliers in time series. *Journal of the Royal Statistical Society. Series B (Methodological)*, 34(3), 350–363. Retrieved July 1, 2014, from <http://www.jstor.org/stable/2985071>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. The MIT Press.
- Hawkins, D. M. (1980). *Identification of outliers*. Chapman & Hall. <https://doi.org/10.1007/978-94-015-3994-4>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735–1780.
- Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and practice* (2nd). OTexts. <https://otexts.com/fpp2/>
- Kejariwal, A., Vallis, O., & Hochenbaum, J. (2015). *Introducing practical and robust anomaly detection in a time series*. Twitter Engineering. Retrieved April 21, 2026, from https://blog.x.com/engineering/en_us/a/2015/introducing-practical-and-robust-anomaly-detection-in-a-time-series

- Kim, T.-Y., & Cho, S.-B. (2018). Web traffic anomaly detection using c-lstm neural networks. *Expert Systems with Applications*, 106, 66–76.
<https://doi.org/https://doi.org/10.1016/j.eswa.2018.04.004>
- Knorr, E. M., & Ng, R. T. (1998). Algorithms for mining distance-based outliers in large datasets. *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*, 392–403.
- Laptev, N., Amizadeh, S., & Flint, I. (2015). Generic and scalable framework for automated time-series anomaly detection. *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
<https://doi.org/10.1145/2783258.2788611>
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest [1550-4786/08 \$25.00 © 2008 IEEE]. *2008 Eighth IEEE International Conference on Data Mining*, 413–422.
<https://doi.org/10.1109/ICDM.2008.17>
- Malhotra, P., Vig, L., Shroff, G. M., & Agarwal, P. (2015). Long short term memory networks for anomaly detection in time series. *Proceedings of the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*.
<https://api.semanticscholar.org/CorpusID:43680425>
- Munir, M., Siddiqui, S. A., Dengel, A., & Ahmed, S. (2019). Deepant: A deep learning approach for unsupervised anomaly detection in time series. *IEEE Access*, 7, 1991–2005.
<https://doi.org/10.1109/ACCESS.2018.2886457>
- NVIDIA Corporation. (2023). *Train with mixed precision* [Consultado en October 5, 2026]. NVIDIA.
<https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>
- Parametrix. (2024, July). *Crowdstrike's impact on the fortune 500: An impact analysis* (Informe Técnico) (Disponible en: www.parametrixinsurance.com). Parametrix Solutions Inc. Retrieved February 14, 2026, from https://cdn.prod.website-files.com/64b69422439318309c9f1e44/66a14199d3c5dde3b1c40aaa_CrowdStrikes%20Impact%20on%20the%20Fortune%20500_%202024%20_Parametrix%20Analysis.pdf
- Peña, D. (2010). *Análisis de series temporales*. Alianza Editorial.
- Ren, H., Xu, B., Wang, Y., Yi, C., Huang, C., Kou, X., Xing, T., Yang, M., Tong, J., & Zhang, Q. (2019). Time-series anomaly detection service at microsoft. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 3009–3017.
<https://doi.org/10.1145/3292500.3330680>
- Shumway, R. H., & Stoffer, D. S. (2025). *Time series analysis and its applications: With r examples* (5th ed.). Springer. <https://doi.org/10.1007/978-3-031-70584-7>
- Siami-Namini, S., Tavakoli, N., & Siami Namin, A. (2018). A comparison of arima and lstm in forecasting time series. *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 1394–1401. <https://doi.org/10.1109/ICMLA.2018.00227>
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley Publishing Company.
- Wong, J., Colburn, C., Meeks, E., & Vedaraman, S. (2015, February). *Rad: Outlier detection on big data*. Netflix TechBlog. Retrieved April 21, 2026, from <https://netflixtechblog.com/rad-outlier-detection-on-big-data-d6b0494371cc>
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into deep learning* [<https://D2L.ai>]. Cambridge University Press.